Desarrollo de Interfaces · para leer en el móvil

19-10-2026. Toda la asignatura en un solo PDF: primero lo que estáis viendo ahora, luego lo anterior y al final la guía completa.

Por dónde vas

- **Ahora en clase: Supabase + Angular**: login, perfiles con roles y rutas protegidas (dossier del profe `Dossier_Tecnico_Supabase_y_Angular.pdf`).
- **Lo anterior (9 y 10 de octubre):** Reactividad con Signals, con el carrito (04) y el TechTicket (05). Están entregadas, y el servicio de Supabase usa justo ese patrón.

Orden para hoy:

#	Qué leer	En este PDF	En tu PC (`C:\DAM 2\Desarrollo de Interfaces\`)
1	**Supabase**: qué es, el SQL, el servicio, el guard y lo que falta	Parte 1 →	`06-Supabase-Auth-Roles\` (por crear)
2	Signals: `signal`, `computed`, `effect` (el carrito)	Parte 2 →	`04-Reactividad-Signals-Carrito\00-teoria-y-apuntes.md`
3	Store con Signals + `HttpClient` + Android	Parte 3 →	`05-Lab-Reactividad-TechTicket\00-teoria-y-apuntes.md`
4	Solo si algo no te cuadra: §3.10, §5.3, §5.6, §5.8, §5.9 y §15.3	Parte 4 →	`00-Apuntes\01-guia-de-estudio-desde-cero.md`

Si en Supabase te suena raro lo de `signal` y `computed` públicos y privados, lee antes la Parte 3 (10 minutos).

Los PDF del profe van aparte: el dossier de Supabase, `Dossier_Reactividad_y_Signals.pdf` (carpeta 04) y `Lab_Reactividad.pdf` (carpeta 05). Las páginas de cada cosa están en la tabla "Dónde está la teoría" de cada parte.

Pendiente

- **06:** crear la carpeta `06-Supabase-Auth-Roles`, meter el dossier y montar el proyecto (Parte 1, pasos 0 a 5).
- **05:** abrir el proyecto en Android Studio y probarlo en el emulador (`npx cap open android`).
- **GitHub:** las carpetas 04 y 05 no están subidas (Git las ve como nuevas). Falta `git add`, `commit` y `push`. El `README.md` de la asignatura solo lista hasta la práctica 02.

PARTE 1 · LO DE AHORA

06 · Supabase + Angular: login, perfiles y roles (lo de ahora en clase)

Dónde está la teoría

Qué	Dónde
Dossier del profe	`Dossier_Tecnico_Supabase_y_Angular.pdf` (8 págs.)
· Qué es Supabase, PostgREST, GoTrue y RLS	Dossier, pág. 2
· Realtime y Storage	Dossier, pág. 3
· El SQL: tipo `app_role` y tabla `profiles`	Dossier, pág. 3
· RLS, `has_role()`, políticas y trigger	Dossier, págs. 4–5
· `npm install` y `environment.ts`	Dossier, pág. 5
· El servicio `supabase.service.ts`	Dossier, págs. 6–8
· El guard `auth.guard.ts`	Dossier, pág. 8
Signals y patrón Store (lo que usa el servicio)	Partes 2 y 3 de este PDF (carrito y TechTicket)
Tu guía de estudio	Parte 4 → §3.10 (`async/await`), §5.3 (`[(ngModel)]` para el login), §5.8 (rutas), §5.9 (`inject`), §15.3 (por qué con `HttpClient` NO se usa `async/await`)

Carpeta en tu PC: todavía no existe. Créala siguiendo tu esquema: `C:\DAM 2\Desarrollo de Interfaces\06-Supabase-Auth-Roles\` y mete ahí el dossier y el proyecto.

La idea en 3 líneas

- **Supabase es un backend ya hecho**: base de datos PostgreSQL, API REST que se genera sola, login de usuarios, tiempo real y almacenamiento de ficheros. Tú solo haces el frontend (Ionic + Angular).

- Es la **alternativa a Firebase**, pero **relacional**: tablas, SQL, claves foráneas y transacciones (ACID), no documentos NoSQL.
- **La seguridad vive en la base de datos** (RLS), no en la app. Por eso la clave que pones en el frontend puede ser pública.

Con lo que ya conoces

Ya lo has usado	En Supabase
SQLite del proyecto `03-JDBC-SQLite-CRUD` (un fichero en tu PC)	PostgreSQL en la nube, con su servidor
Escribir tú `SELECT ... WHERE id = ?` con JDBC	PostgREST: escribes `.from('profiles').select(...).eq('id', x)` y él genera el SQL
dummyjson o jsonplaceholder (la API de otro, solo para leer)	Una API REST de **tus** tablas, sin programar el backend
`if (!usuario.esAdmin()) throw ...` en tu Service de Java	RLS: la regla está en la BD y se comprueba **en cada fila**
`TicketService` con signals (actividad 05)	`SupabaseService`: el mismo patrón Store
`DriverManager.getConnection(url, usuario, pass)`	`createClient(url, key)` te da el objeto cliente

Los 5 servicios de Supabase (dossier, págs. 2–3)

1. PostgREST: la API que se hace sola

Mira las tablas de tu esquema `public` y crea automáticamente los endpoints REST para hacer CRUD. Tú no programas ningún backend.

La misma consulta de tres formas:

```
// Supabase (Angular)
supabase.from('profiles')
  .select('id, email, role')
  .eq('id', userId)
  .single();
```

```
-- Lo que acaba ejecutando PostgreSQL
SELECT id, email, role FROM profiles WHERE id = :userId;
```

```
// Lo mismo en Java con JDBC (Acceso a Datos)
PreparedStatement ps = con.prepareStatement(
  "SELECT id, email, role FROM profiles WHERE id = ?");
```

```
ps.setString(1, userId);
ResultSet rs = ps.executeQuery();
```

- `.eq('id', x)` es el `WHERE id = x`. Hay más filtros: `.neq`, `.gt`, `.like`, `.order`...
- `.single()` significa "espero **una sola fila**": te devuelve un objeto y no un array. Si salen 0 filas o más de 1, devuelve error.
- Te devuelve el resultado en JSON, como dummyjson.

2. GoTrue (Auth): el login

Gestiona los usuarios: registro, login con contraseña, login con Google o GitHub (OAuth) y *magic links* (un enlace al correo para entrar sin contraseña).

Cuando entras te da un **JWT** (*JSON Web Token*). **Analogía: la pulsera de un festival.** En la puerta (el login) enseñas la entrada una vez y te ponen la pulsera; después, en cada barra (cada petición), solo enseñas la pulsera. El JWT lleva dentro tu id (`sub`), tu email y metadatos, firmado para que nadie lo pueda falsificar.

Los usuarios se guardan en una tabla interna, `auth.users`, que tú no tocas.

3. RLS (Row Level Security): el portero está en la base de datos

Con RLS activado, PostgreSQL comprueba **cada fila** de cada `SELECT`, `INSERT`, `UPDATE` y `DELETE` contra unas reglas (las **políticas**).

- `auth.uid()` devuelve el id del usuario que hace la petición (lo saca del JWT).
- **Analogía:** es como si la BD añadiera un `WHERE` invisible a cada consulta. Tú pides "todos los perfiles" y te devuelve solo el tuyo, porque la política dice `auth.uid() = id`.

Por qué importa: la clave del frontend es pública (cualquiera la ve con F12). Da igual: aunque alguien la use, la BD solo le enseña lo que las políticas le permiten.

4. Realtime: la BD te avisa

Abre un **WebSocket** y te manda un evento cada vez que hay un `INSERT`, `UPDATE` o `DELETE` en una tabla. Es como un Observable que nunca se completa: te vas enterando de los cambios sin volver a preguntar. Sirve para chats y paneles en directo.

5. Storage: ficheros

Guarda imágenes y documentos (avatares, PDFs…), con carpetas protegidas también por políticas RLS.

Conceptos nuevos en una línea

Concepto	Qué es
BaaS	*Backend as a Service*: el backend te lo da otro ya montado (Supabase, Firebase)
ACID	Las transacciones se hacen enteras o no se hacen; la BD nunca se queda a medias
JWT	La "pulsera": un token firmado con quién eres
RLS / política	Regla en la BD que decide qué filas puede ver o tocar cada usuario
Trigger	Código de la BD que se ejecuta solo cuando pasa algo (como un `effect()`, pero en la BD)
UUID	Id largo y aleatorio (`3f2a...`) en vez de 1, 2, 3
ENUM	Lista cerrada de valores, igual que un `enum` de Java
Clave publishable / `anon`	Clave pública para el frontend. Puede estar en el código
Clave secret / `service_role`	Clave que **se salta RLS**. Nunca en el frontend ni en GitHub
Guard	Función que decide si puedes entrar en una ruta de Angular
Sesión persistente	La sesión queda guardada: cierras la app, vuelves y sigues dentro

---

El proyecto: qué vas a montar

Al final tienes una app Ionic donde:

1. El usuario entra con **email y contraseña**.

2. Al registrarse se le crea solo un **perfil** con rol `viewer`.
3. Hay 3 roles: `admin`, `developer` y `viewer`.
4. Algunas rutas solo se abren con cierto rol. Si no lo tienes, te manda a `/unauthorized`.
5. Un usuario solo ve **su** perfil; el admin los ve **todos** y es el único que puede cambiar roles.
6. La sesión se queda guardada.

El viaje de los datos:

```
Login (email + pass)
   |
   ▼
GoTrue ──► te da el JWT ──► se guarda (persistSession)
   |
   ▼
onAuthStateChange avisa al SupabaseService
   |
   ├─ currentUserSignal.set(user)
   └─ fetchUserProfile()
         |   .from('profiles')...single()
         ▼
      PostgreSQL + RLS (¿auth.uid() = id o admin?)
         |
         ▼
      currentProfileSignal.set(perfil)
         |
         ▼
computed: isAuthenticated / isAdmin / profile
         |
         ▼
roleGuard deja pasar (o te manda a /unauthorized)
```

Paso 0 · Crear el proyecto en Supabase (el dossier no lo cuenta)

1. En supabase.com: **New project**. Ponle nombre, contraseña de la BD y región en Europa.
2. Busca la **Project URL** y la **clave pública** (Project Settings → API Keys, o el botón **Connect**).
   - Ahora el panel la llama **publishable key** (empieza por `sb_publishable_...`). La antigua `anon` todavía funciona y va en el mismo sitio, pero Supabase la retira a finales de 2026.
   - **Nunca** copies la `secret` / `service_role`.

3. En **Authentication**, comprueba que el proveedor Email está activado. Para pruebas en clase te viene bien desactivar la confirmación por correo.

Paso 1 · El SQL, bloque a bloque (dossier, págs. 3–5)

Se pega en el **SQL Editor** del panel de Supabase.

1. El ENUM de roles. Igual que un `enum` de Java: solo admite esos 3 valores.

```
CREATE TYPE public.app_role AS ENUM ('admin', 'developer', 'viewer');
```

2. La tabla `profiles`. `auth.users` es la tabla interna de Supabase (no se toca); `profiles` es **tu** tabla con lo que quieras guardar de cada usuario, en relación 1 a 1.

```
id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
role public.app_role NOT NULL DEFAULT 'viewer',
```

- `REFERENCES auth.users(id)` es una **clave foránea**: el id del perfil tiene que existir en `auth.users`.
- `ON DELETE CASCADE`: si borras el usuario, su perfil se borra solo.
- `DEFAULT 'viewer'`: si no dices rol, eres `viewer`.

3. Activar RLS.

```
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
```

A partir de aquí, **sin políticas nadie puede leer ni escribir** la tabla desde la API: todo está denegado por defecto. Las políticas abren puertas concretas.

4. La función `has_role()`. Devuelve `true` si el usuario actual tiene el rol que le pides.

```
SELECT EXISTS (
  SELECT 1 FROM public.profiles
  WHERE id = auth.uid() AND role = required_role
);
```

- `EXISTS (...)` da `true` si la subconsulta devuelve alguna fila.

- `SECURITY DEFINER` (lo importante): ejecuta la función con los permisos de quien la creó y **sin pasar por RLS**. Si no lo pusieras, la política de `profiles` llamaría a `has_role()`, que lee `profiles`, que vuelve a mirar la política... y entra en bucle infinito.

5. Las políticas.

```
-- SELECT: tu perfil, o todos si eres admin
USING ( auth.uid() = id OR public.has_role('admin') )

-- UPDATE: solo admin
USING (public.has_role('admin'))
WITH CHECK (public.has_role('admin'))
```

- `USING` = qué filas **existentes** puedes tocar. Es un filtro, como un `WHERE`.
- `WITH CHECK` = cómo tiene que quedar la fila **después** del cambio. Es una validación.
- No hay política de `INSERT`: los perfiles solo los crea el trigger.

6. El trigger `handle_new_user`. Cuando alguien se registra (entra una fila en `auth.users`), se crea su perfil automáticamente.

```
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

- `NEW` es la fila recién insertada: `NEW.id`, `NEW.email`.
- `->>` saca un campo de un JSON como texto: `raw_user_meta_data->>'full_name'`.
- `COALESCE(a, b)` devuelve `a` y, si es `null`, `b`. Es el `??` de TypeScript (en Java, `Objects.requireNonNullElse(a, b)`).
- **Por qué el rol se lee de `app_metadata` y no de `user_metadata`:** `user_metadata` lo rellena el propio usuario al registrarse, y si el rol saliera de ahí cualquiera se haría admin. `app_metadata` solo lo puede escribir el servidor. Por eso, por defecto, todos entran como `viewer`.

Tu primer admin. Créate un usuario (Authentication → Users → Add user) y luego, en el SQL Editor (que funciona como superusuario y no pasa por RLS):

```sql
UPDATE public.profiles SET role = 'admin'
WHERE email = 'tu-correo@ejemplo.com';
```

Paso 2 · Instalar y configurar (dossier, pág. 5)

```
npm install @supabase/supabase-js
```

En `src/environments/environment.ts` van la URL y la clave pública. Si tu proyecto no tiene la carpeta `environments`, créala con `ng generate environments`.

La clave puede ir en el código y subirse a GitHub porque es **pública**: quien protege los datos es RLS.

Paso 3 · El servicio `SupabaseService`, bloque a bloque (dossier, págs. 6–8)

Es el **mismo patrón Store** que `TicketService` (actividad 05):

`TicketService` (05)	`SupabaseService` (06)
`private state = signal(...)`	`private currentUserSignal` y `currentProfileSignal`
`public tickets = computed(...)`	`public user`, `profile`, `isAuthenticated` e `isAdmin` (todos `computed`)
`inject(HttpClient)`	`createClient(url, key)`
`http.get().subscribe(...)` (Observable)	`await supabase.from(...)` (Promise)

Las signals públicas:

```
isAuthenticated = computed(() => !!this.currentUserSignal());
isAdmin = computed(() => this.currentProfileSignal()?.role === 'admin');
```

- `!!x` convierte cualquier cosa a boolean: `null` da `false` y un objeto da `true`. En Java sería `user != null`.
- `?.` (*optional chaining*): si el perfil es `null`, devuelve `undefined` en vez de petar. En Java: `profile != null && profile.getRole().equals("admin")`.

`createClient` y sus opciones:

- `persistSession: true`: guarda la sesión en el navegador; cierras y vuelves y sigues dentro.
- `autoRefreshToken: true`: renueva el JWT antes de que caduque (la pulsera dura un rato).
- `detectSessionInUrl: true`: para el login por enlace o con Google, que vuelve con el token en la URL.

`initSessionListener()` tiene 2 pasos:

1. `getSession()`: "¿había ya una sesión guardada?".
2. `onAuthStateChange(...)`: se queda **escuchando** los logins y logouts, como un `subscribe` que no termina.

Destructuring:

```
const { data: { session } } = await this.supabase.auth.getSession();
// Java: Session session = respuesta.getData().getSession();
```

Aquí SÍ hay `async/await`. Con `HttpClient` usabas `subscribe` porque devuelve Observables (guía §15.3). La librería de Supabase devuelve **Promises**, así que se usa `await` (guía §3.10).

`{ data, error }` en vez de excepciones:

```
const { data, error } = await this.supabase.from('profiles')...;
if (error) { ... }
```

Las consultas de Supabase **no lanzan excepciones**: te devuelven el error dentro del resultado y tú lo compruebas con `if`. En `signIn` hacen `if (error) throw error;` para convertirlo en excepción, y así la página de login lo puede capturar con `try/catch`, como en Java.

`fetchUserProfile` es la consulta de PostgREST de arriba. RLS le añade por detrás el `WHERE` invisible (`auth.uid() = id OR has_role('admin')`).

Paso 4 · El guard (dossier, pág. 8)

```
export const roleGuard = (expectedRole: ...): CanActivateFn => {
  return async () => { ... };
};
```

- `CanActivateFn` es una función que Angular ejecuta **antes** de entrar en una ruta. Si devuelve `true`, entras; si devuelve `router.createUrlTree(['/unauthorized'])`, te redirige.
- `roleGuard('admin')` es **una función que devuelve una función**. Es como un método de Java que devuelve una lambda:

```
static Predicate<Usuario> tieneRol(String rol) {
    return u -> u.getRol().equals(rol);
}
```

- Dentro usa `inject()` para pedir el servicio y el router (guía §5.9).

Paso 5 · Lo que el dossier no trae y vas a necesitar (pistas, lo haces tú)

1. **Página de login**: dos `ion-input` con `[(ngModel)]` (guía §5.3) y un botón con `(click)="entrar()"`. Dentro, `try { await this.supabase.signIn(email, pass); navegar } catch (e) { mostrar error }`.
2. **Rutas** en `app.routes.ts` (guía §5.8):

```
{ path: 'login', loadComponent: () => ... },
{ path: 'admin', loadComponent: () => ...,
  canActivate: [roleGuard('admin')] },
{ path: 'unauthorized', loadComponent: () => ... },
```

3. **Página** `/unauthorized`: un mensaje y un botón para volver.
4. **Botón de cerrar sesión** que llame a `signOut()`.
5. **En los templates**, usa las signals: `@if (supabase.isAdmin()) { ... }`. Para eso el servicio tiene que ser `public` en el componente, como en el TechTicket.

⚠️ Avisos antes de copiar el dossier (comprobados)

1. **El guard falla si recargas (F5) estando en una ruta protegida.** Cargar la sesión y el perfil es asíncrono, así que cuando el guard lee `profile()` todavía vale `null` y manda al admin a `/unauthorized`. El comentario dice "esperar a que la señal resuelva", pero el código no espera. **Idea para arreglarlo:** guarda en el servicio la promesa del arranque (`this.ready =`

`this.initSessionListener();` en el constructor) y en el guard haz `await supabaseService.ready;` antes de leer `profile()`.

2. **No hagas `await` de llamadas a Supabase dentro de `onAuthStateChange`.** La documentación de Supabase avisa de un fallo: si dentro de ese callback haces una llamada asíncrona a Supabase, la siguiente llamada se puede quedar colgada para siempre. El dossier lo hace (`processSession` consulta `profiles`). **Solución habitual:** sacarlo del callback con `setTimeout(() => this.processSession(session), 0);` y que el callback no sea `async`.
3. **Detalle sin importancia:** `onAuthStateChange` también salta al arrancar (evento `INITIAL_SESSION`), así que con el `getSession()` el perfil se pide dos veces al abrir la app. No rompe nada.

Comprueba que lo entiendes

1. ¿Por qué se puede poner la clave pública en el frontend sin peligro? → Porque solo identifica el proyecto. Lo que ves depende de tu JWT y de las políticas RLS.
2. ¿Qué pasa si activas RLS y no creas ninguna política? → Que nadie puede leer ni escribir esa tabla desde la API.
3. ¿Por qué el rol se lee de `app_metadata` y no de `user_metadata`? → Porque `user_metadata` lo escribe el propio usuario y se podría poner admin.
4. ¿Por qué aquí sí `async/await` y con `HttpClient` no? → Supabase devuelve Promises; `HttpClient` devuelve Observables.
5. ¿Qué hace `.single()`? → Exige una sola fila y te devuelve un objeto en vez de un array.
6. ¿Diferencia entre `USING` y `WITH CHECK`? → `USING` filtra qué filas puedes tocar; `WITH CHECK` valida cómo quedan después del cambio.
7. ¿Para qué sirve `SECURITY DEFINER` en `has_role()`? → Para que no pase por RLS y no entre en bucle (una política de `profiles` que lee `profiles`).
8. ¿Qué hace el trigger? → Al registrarse un usuario, le crea su fila en `profiles` con rol `viewer`.

PARTE 2

04 · Reactividad con Signals: el carrito (Seguimiento, entrega 10-10-2026)

Dónde está la teoría

Qué	Dónde
Dossier del profe	`Dossier_Reactividad_y_Signals.pdf` (en esta carpeta)
· Cómo se hacía antes (Zone.js, RxJS)	Dossier, pág. 1
· Qué es una signal y sus 3 tipos	Dossier, pág. 2
· El código del carrito	Dossier, págs. 3–5
· `OnPush` y por qué se usa `update()` con `...`	Dossier, pág. 6
· `computed()` y `effect()` por dentro	Dossier, pág. 7
· `@for` y `track`	Dossier, pág. 8
Tu guía de estudio	`00-Apuntes/01-guia-de-estudio-desde-cero.md` → §5.3 (binding `{{ }}` y `(click)`), §5.4 (`@for`, `@if`), §5.6 (Signals)

La idea en 3 líneas

- **`signal()`**: una caja con un valor que **avisa** cuando cambia.
- **`computed()`**: un valor **calculado** a partir de otras signals. Se recalcula solo, y solo cuando cambian.
- **`effect()`**: un trozo de código que **se ejecuta solo** cada vez que cambian las signals que lee (para cosas "de fuera": `console.log`, guardar en `localStorage`...).

Analogía con Excel: `A1` es una signal, `B1 = SUMA(A1:A9)` es un computed y una macro que salta cada vez que cambias `A1` es un effect.

El código, bloque a bloque

Archivo: `carrito-signals/src/app/carrito-signals/carrito-signals.component.ts`

1. El estado: `signal`

```
public productos = signal<Producto[]>([
  { id: 1, nombre: 'Licencia IDE', precio: 89 },
  { id: 2, nombre: 'Teclado Mecánico', precio: 120 }
]);
```

- **Leer**: `this.productos()`, con paréntesis, porque una signal es una función.
- **Escribir**: `.set(valorNuevo)` lo sustituye entero; `.update(anterior => nuevo)` lo calcula a partir del anterior.

2. El valor derivado: `computed`

```
public precioTotal = computed(() => {
  return this.productos().reduce((total, prod) => total + prod.precio, 0);
});
```

- `reduce` recorre el array acumulando: empieza en `0` y en cada vuelta suma el precio del producto (0 → 89 → 209).
- Angular apunta que `precioTotal` **depende** de `productos`. Si `productos` no cambia, no vuelve a hacer el `reduce` y usa el valor guardado (caché).

3. El efecto secundario: `effect`

```
constructor() {
  effect(() => {
    console.log(`[Persistencia] ... Items: ${this.productos().length}`);
  });
}
```

- Se ejecuta **una vez al arrancar** (Items: 2) y **otra vez cada vez que cambia** `productos` (Items: 3). Eso es la captura 3.
- Va en el constructor porque ahí Angular lo ata a la vida del componente: cuando el componente se destruye, el efecto se para solo y no hay fugas de memoria.

4. Cambiar el estado: `update` con `...` (spread)

```
this.productos.update(anterior => [...anterior, nuevoProducto]);   // añadir
this.productos.update(anterior => anterior.filter(p => p.id !== id)); //
  eliminar
```

- `[...anterior, nuevo]` crea un **array nuevo** con todo lo de antes más el nuevo. `filter` también devuelve un array nuevo.
- **Por qué no `push()`**: `push` mete el producto en el **mismo** array (misma referencia en memoria) y Angular cree que no ha cambiado nada, así que la pantalla no se repinta.

5. `ChangeDetectionStrategy.OnPush`

Le dice a Angular que solo revise este componente si hay un evento suyo (un `click`) o si cambia una signal que usa el template. Así no recorre toda la app en cada evento.

6. El template

```
Total: {{ precioTotal() }}€                          <!-- interpolación de un
  computed -->

@for (prod of productos(); track prod.id) {        <!-- repetir por cada
  producto -->
  ... {{ prod.nombre }} ... (click)="eliminarProducto(prod.id)"
} @empty {
  El carrito está vacío                             <!-- si el array está
  vacío -->
}
```

- `track prod.id` es obligatorio. Le dice a Angular cómo reconocer cada fila, para que al añadir o quitar uno solo toque **ese** nodo del DOM y no repinte la lista entera.

Qué demuestra cada captura (`capturas/`)

Captura	Qué se ve	Qué concepto demuestra
1 · render inicial	2 productos, Total 209 €	`signal` inicial + `computed` (89 + 120)
2 · producto añadido	Aparece "Monitor ###"	`update` con spread + `@for track`
3 · consola	`Items: 2` y luego `Items: 3`	`effect` al arrancar y al cambiar

Captura	Qué se ve	Qué concepto demuestra
4 · total	Total 459 €	`computed` recalculado (89 + 120 + 250)

Cosas que pasaron montándolo (por si te pasan a ti)

- **Si pones el componente directamente dentro de `<ion-app>` se ve mal** (solo sale un producto y no se ve el botón). Ionic necesita que la página esté dentro de `<ion-router-outlet>`, que le pone la clase `ion-page`. Por eso `app.ts` tiene `<ion-router-outlet>` y `app.routes.ts` carga el componente con `loadComponent`, como hace la plantilla `ionic start blank`.
- **Versión de Ionic**: el dossier importa de `'@ionic/angular/standalone'`. Eso existe en **Ionic 8** (el que usa este proyecto, `^8.8.19`). En **Ionic 9**, que es el que tienes en tus otros proyectos, se importa de `'@ionic/angular'` y `/standalone` ya no existe.

Cómo ejecutarlo

```
cd carrito-signals
npm install
npm start          # abre http://localhost:4200
```

Abre la consola del navegador (F12 → Console) para ver los mensajes del `effect`.

Comprueba que lo entiendes

1. ¿Qué diferencia hay entre `.set()` y `.update()`? → `set` pone un valor nuevo sin mirar el anterior; `update` recibe el anterior y devuelve el nuevo.
2. ¿Por qué `this.productos().push(x)` no actualizaría la pantalla? → Porque es el mismo array (misma referencia) y la signal no detecta el cambio.
3. ¿Cuántas veces se ejecuta el `reduce` de `precioTotal` si nadie toca `productos`? → Una. Después usa el valor cacheado.

4. ¿Cuándo se ejecuta el `effect`? → Al crear el componente y cada vez que cambia una signal que lee dentro.
5. ¿Para qué sirve `track prod.id`? → Para que Angular identifique cada fila y solo añada o quite la que cambia.

PARTE 3

05 · Laboratorio TechTicket: estado reactivo (Store + Signals) y Android (entrega 09-10-2026)

Dónde está la teoría

Qué	Dónde
Enunciado del profe	`Lab_Reactividad.pdf` (en esta carpeta)
· Comandos para crear el proyecto	Lab, pág. 1
· Patrón Store / MVVM y SOLID	Lab, pág. 2
· El servicio `TicketService`	Lab, págs. 3–4
· El componente `TicketDashboard`	Lab, págs. 5–7
· Capacitor y Android	Lab, pág. 7
Signals (`signal`, `computed`, `update`)	`../04-Reactividad-Signals-Carrito/00-teoria-y-apuntes.md` (léelo antes que este)
Tu guía de estudio	`00-Apuntes/01-guia-de-estudio-desde-cero.md` → §5.7 (`ngOnInit`), §5.9 (servicios e inyección de dependencias), §14 (qué es una API REST), §15.4 (`subscribe` paso a paso), §15.7 (`provideHttpClient`)

La idea

Es el carrito pero "en serio": el estado ya no vive en el componente, sino en un **servicio** (el *Store*). El componente solo pinta lo que el servicio le da.

```
ngOnInit ──► cargarDatos() ──► ticketService.fetchTickets()
                                    │
                                    ├─ state.update(loading: true)       → se
ve el spinner
                                    ├─ http.get('jsonplaceholder/todos')
                                    │     ├─ next  → state.update(data,
```

```
loading: false) → se ve la lista
                                  |     └─ error → state.update(error,
loading: false) → mensaje + Reintentar
                                  ▼
                computed tickets / isLoading / errorMessage  (solo lectura)
                                  ▼
                componente: computed ticketsFiltrados (datos + filtro) →
template
```

El servicio, bloque a bloque

Archivo: `tech-ticket/src/app/core/services/ticket.service.ts`

```
private http = inject(HttpClient);
```

Pide el `HttpClient` a Angular en vez de hacer `new`. Esto es la **inyección de dependencias** (DIP). Para que funcione, `app.config.ts` tiene `provideHttpClient()`.

```
private state = signal<TicketState>({ data: [], loading: false, error: null
  });
```

Un solo objeto con todo el estado: los datos, si está cargando y el error. Es `private`, así que nadie de fuera puede tocarlo.

```
public tickets      = computed(() => this.state().data);
public isLoading    = computed(() => this.state().loading);
public errorMessage = computed(() => this.state().error);
```

Lo que se enseña fuera es **solo lectura**: un `computed` no tiene `.set()`. Esto es el **OCP** (abierto a añadir selectores nuevos, cerrado a que te cambien el estado desde fuera).

```
this.state.update(s => ({ ...s, loading: true, error: null }));
```

`{ ...s, loading: true }` es el spread de **objetos**: copia todo lo que había en `s` y sobreescribe `loading` y `error`. Es la misma idea que `[...array, nuevo]` del carrito: siempre un objeto nuevo.

```
this.http.get<Ticket[]>(url).subscribe({
  next:  (response) => this.state.update(s => ({ ...s, data: response,
   loading: false })),
  error: ()         => this.state.update(s => ({ ...s, error: 'Error de
   red…', loading: false }))
});
```

`get` devuelve un Observable y no hace nada hasta que te suscribes (guía §15.4). `next` se ejecuta si va bien y `error` si falla.

El componente, bloque a bloque

Archivo: `tech-ticket/src/app/features/ticket-dashboard/ticket-dashboard.component.ts`

- `public ticketService = inject(TicketService);` es **public** para poder usarlo en el template (`ticketService.isLoading()`).
- `filtroActual = signal<'todos' | 'abiertos' | 'cerrados'>('todos')` usa un **tipo unión**: solo admite esos 3 textos.
- `ticketsFiltrados = computed(...)` depende de **dos** signals, `tickets()` y `filtroActual()`. Cambia una cualquiera y se recalcula.
- `ngOnInit()` se ejecuta una vez al crear el componente y lanza la carga (guía §5.7).
- `cambiarFiltro(event)`: el `<ion-segment>` lanza `ionChange` y el valor elegido viene en `event.detail.value`.

Template:

- `@if (isLoading()) {…} @else if (errorMessage()) {…} @else {…}` son los 3 estados de la pantalla: cargando, error y datos.
- `[color]="ticket.completed ? 'success' : 'warning'"` es un property binding con un ternario: verde si está resuelto y amarillo si está pendiente.

SOLID en una tabla

Principio	Dónde se ve
SRP (una responsabilidad)	El servicio solo gestiona el estado; el componente solo pinta y recoge clicks
OCP (abierto/cerrado)	`state` privado + `computed` públicos de solo lectura
DIP (inversión de dependencias)	`inject(HttpClient)` e `inject(TicketService)`, nunca `new`

Android con Capacitor (capturas 6 y 7)

```
ionic build --prod        # compila a www/
npx cap add android       # crea la carpeta android/ (una sola vez)
npx cap sync android      # copia www/ dentro de android/ (cada vez que
  vuelves a hacer build)
npx cap open android      # abre Android Studio → emulador   (PENDIENTE)
npx cap run android       # o directo a un móvil por USB
```

- **Cosa que falló y cómo se arregló**: `ionic build` daba `Project "app" does not exist`. El Ionic CLI busca un proyecto llamado `app` en `angular.json`. Se renombró a `app` y se puso `outputPath: www` (lo que hace `ionic start`), y en `capacitor.config.ts` → `webDir: 'www'`.
- **Pendiente**: abrirlo en Android Studio y probarlo en el emulador.

Qué demuestra cada captura (`capturas/`)

Captura	Qué se ve
1_cargando	Spinner "Sincronizando incidencias..." (`isLoading`)
2_todos	Los 15 tickets con badge Pendiente/Resuelto
3_abiertos	Solo los 8 con `completed = false`
4_cerrados	Solo los 7 con `completed = true`
5_error_red	Mensaje de error + botón Reintentar
6_terminal_build	`ionic build --prod`
7_terminal_capacitor	`npx cap add android` + `npx cap sync android`

Las capturas se hicieron en un entorno sin acceso a jsonplaceholder, así que la app recibió la misma respuesta que da la API real (los 15 primeros todos). En tu PC va contra la API de verdad.

Cómo ejecutarlo

```
cd tech-ticket
npm install
ionic serve        # o npm start
```

Comprueba que lo entiendes

1. ¿Por qué `state` es `private` y `tickets` es un `computed` público?
   → Para que solo el servicio pueda cambiar el estado; desde

fuera solo se lee (OCP y encapsulación).

2. ¿Qué hace `{ ...s, loading: true }`? → Crea un objeto nuevo copiando `s` y cambiando solo `loading`.
3. ¿Por qué al cambiar de filtro no se vuelve a llamar a la API? → Porque `ticketsFiltrados` filtra en memoria los datos que ya están en la signal.
4. ¿Qué tiene que pasar para que se vea el botón Reintentar? → Que la petición falle: el `subscribe` entra en `error` y se rellena `errorMessage`.
5. ¿Cuándo hay que repetir `npx cap sync android`? → Cada vez que cambias el código y vuelves a hacer `ionic build`.

PARTE 4 · GUÍA DE ESTUDIO COMPLETA

Desarrollo de Interfaces — Guía de estudio desde cero

> Para leer en VS Code con la vista previa: abre este fichero y pulsa **Ctrl + Shift + V**. Tiempo estimado: 2–3 horas leyendo con calma y abriendo el código a la vez. Si algo no lo entiendes, sigue leyendo: casi todo se aclara con el ejemplo siguiente.

Índice

1. El mapa: de qué va la asignatura — **incluye qué es un framework y qué problema resuelve Angular**
2. Las herramientas: Node, npm, Ionic CLI, la terminal
3. JavaScript y TypeScript: lo mínimo para leer el código
4. Práctica: paradigmas asíncronos de JavaScript (`paradigmas-js`)
5. Angular: las piezas — **data binding explicado desde cero en 5.3**
6. Práctica 1: el componente galería (data binding)
7. Práctica: `usersApp` línea a línea
8. Práctica 2: `corporateApp` línea a línea
9. Cómo harías la Corporate App tú solo, desde cero
10. Git, GitHub y Vercel
11. Glosario
12. Ejercicios para practicar (con soluciones)
13. Chuleta para mañana en clase

Parte 2 — Consumo de APIs REST (clase del 05-10-2026)

14. ¿Qué es una API y qué es REST?
15. Promise vs Observable (`HttpClient` y RxJS) — **por qué con APIs no se usa `async/await`**
16. Actividad guiada: `dam2-productos` (dummyjson) línea a línea
17. El ejemplo del dossier: directorio de usuarios con `async` pipe
18. Ejercicio propuesto: Maestro-Detalle con Rick and Morty
19. Preguntas de repaso de la Parte 2

Parte 3 — Tu proyecto `dam2-productos` y Git a fondo (06-10-2026)

20. Tu proyecto `dam2-productos` (actividad 03) línea a línea
21. Git y GitHub a fondo: ramas, comandos y desde dónde lanzarlos

---

1. El mapa: de qué va la asignatura

Desarrollo de Interfaces trata de hacer **la parte que ve y toca el usuario** de una aplicación: pantallas, botones, listas, formularios, navegación entre pantallas...

En esta asignatura no se hacen interfaces con Java (como Swing o JavaFX), sino con **tecnologías web**, para que **la misma app funcione en el navegador y en el móvil**. A esto se le llama **app híbrida**.

Las capas, de abajo arriba

Piensa en una tarta de capas. Cada una se apoya en la anterior:

```
┌──────────────────────────────────────────────┐
│ Capacitor   → convierte la web en app Android/iOS y da acceso a GPS, cámara...
├──────────────────────────────────────────────┤
│ Ionic       → piezas visuales con aspecto de móvil (<ion-button>, <ion-list>...)
├──────────────────────────────────────────────┤
│ Angular     → organiza la app: componentes, rutas, servicios
├──────────────────────────────────────────────┤
│ TypeScript  → el lenguaje: JavaScript + tipos
├──────────────────────────────────────────────┤
│ HTML + CSS + JavaScript → lo que entiende el navegador
└──────────────────────────────────────────────┘
```

Capa	Qué es	Comparación con lo que ya sabes
HTML	Describe *qué hay* en la página: un título, un botón, una imagen.	Como el "esqueleto".
CSS (o SCSS)	Describe *cómo se ve*: colores, tamaños, márgenes.	La "ropa".
JavaScript	Describe *qué hace*: reaccionar a clics, pedir datos...	El "cerebro".
TypeScript	JavaScript con tipos (`number`, `string`...). Se traduce a JavaScript antes de ejecutarse.	Como Java: te obliga a decir el tipo y te avisa de errores **antes** de ejecutar.

Capa	Qué es	Comparación con lo que ya sabes
Angular	Un *framework*: un esqueleto de proyecto con reglas para organizarlo.	Como Spring para Java: impone estructura.
Ionic	Una librería de componentes visuales con estilo de app móvil.	Como tener los botones y listas de Android ya hechos.
Capacitor	Empaqueta la web como app nativa y le da acceso al hardware.	El "traductor" a móvil.

Idea clave: cuando ejecutas `ionic serve`, todo se **compila** a HTML + CSS + JavaScript normal, que es lo único que el navegador entiende.

1.1 ¿Qué es un framework? (y en qué se diferencia de una librería)

Esta es la pregunta clave para entender Angular, así que vamos despacio.

Librería = una **caja de herramientas**. Tú escribes tu programa como quieras y, cuando necesitas algo, **tú llamas** a la herramienta.

```
// Java: usas la librería Math cuando TÚ quieres
double r = Math.sqrt(16);
```

Tú tienes el control: decides qué se ejecuta, cuándo y en qué orden. La librería solo hace lo que le pides, cuando se lo pides.

Framework = un **esqueleto de programa que ya funciona**, con unas reglas, y **tú rellenas los huecos**. Aquí el control es al revés: **el framework llama a tu código** cuando a él le toca. A esto se le llama **inversión de control**, y se resume en una frase famosa: *"no nos llames, ya te llamaremos nosotros"*.

El mejor ejemplo lo conoces ya: Unity.

```
public class Jugador : MonoBehaviour {
    void Start()  { /* se ejecuta al aparecer el objeto */ }
    void Update() { /* se ejecuta en cada frame */ }
}
```

Tú **nunca** escribes `jugador.Update()` en ningún sitio. Escribes el método y **Unity decide cuándo llamarlo** (60 veces por segundo).

Tampoco escribes el bucle del juego, ni el código que pinta en pantalla, ni el que lee el teclado: eso ya lo hace Unity. Tú solo rellenas lo que es propio de tu juego. **Unity es un framework.**

Angular es exactamente igual, pero para aplicaciones web:

	Unity	Angular
Tú escribes...	Scripts (clases C#)	Componentes (clases TypeScript + HTML)
El framework llama a...	`Start()`, `Update()`	`constructor`, `ngOnInit()`...
El framework se encarga de...	Bucle del juego, pintar, físicas, input	Pintar la página, actualizarla cuando cambian los datos, navegar entre pantallas
Reglas que te impone	Heredar de `MonoBehaviour`, carpeta `Assets`...	`@Component`, carpetas, ficheros `.ts` / `.html` / `.scss`...

Otra analogía: una **librería** es ir a Leroy Merlin a comprar ladrillos y herramientas; construyes la casa como quieras. Un **framework** es una casa prefabricada con la estructura, la luz y el agua ya puestas: tú decides la distribución y la decoración, pero **respetando su estructura**.

Ejemplos	Librería	Framework
Java	`java.util`, Gson, JDBC	Spring, JavaFX
JavaScript	RxJS, Capacitor	**Angular**, React*
Juegos	—	**Unity**

*React técnicamente se llama a sí mismo "librería", pero eso da igual ahora.

> **Para clase:** *"Un framework es una estructura base que impone una forma de organizar el código y que llama a nuestro código cuando corresponde (inversión de control). Una librería es un conjunto de funciones que nosotros llamamos cuando queremos."*

1.2 ¿Qué problema resuelve Angular? (cómo sería SIN Angular)

Imagina que quieres mostrar la lista de productos en una web **solo con HTML y JavaScript**, sin Angular. Tendrías que hacer **a mano** todo esto:

```
<!-- index.html -->
<ul id="lista"></ul>
<p>Total: <span id="total"></span></p>
<button id="boton">Añadir producto</button>
```

```
// codigo.js — SIN Angular
let productos = ['Portátil', 'Monitor'];

function pintar() {
  const ul = document.getElementById('lista');    // 1. buscar el elemento en la página
  ul.innerHTML = '';                              // 2. borrar lo que había
  for (const p of productos) {                    // 3. crear un <li> por producto
    const li = document.createElement('li');
    li.textContent = p;
    ul.appendChild(li);
  }
  document.getElementById('total').textContent = productos.length;   // 4. actualizar el total
}

document.getElementById('boton').addEventListener('click', () => {  // 5. escuchar el clic
  productos.push('Teclado');
  pintar();                                       // 6. ¡acordarte de volver a pintar!
});

pintar();
```

Funciona, pero fíjate en el problema: **cada vez que cambian los datos, tienes que buscar los elementos de la página y cambiarlos tú a mano**, y si se te olvida llamar a `pintar()`, la pantalla muestra datos viejos. Con 4 pantallas, formularios y datos que llegan de internet, esto se vuelve inmanejable.

Lo mismo CON Angular:

```
// lista.page.ts
export class ListaPage {
  productos = signal(['Portátil', 'Monitor']);
  anadir() { this.productos.update(lista => [...lista, 'Teclado']); }   // lista nueva = lo que había + 'Teclado'
}
```

```
<!-- lista.page.html -->
<ul>
  @for (p of productos(); track p) { <li>{{ p }}</li> }
</ul>
<p>Total: {{ productos().length }}</p>
<button (click)="anadir()">Añadir producto</button>
```

Ya no hay `getElementById`, ni `createElement`, ni `pintar()`. **Tú solo describes cómo tiene que verse la página en función de tus datos**, y Angular se encarga de que la pantalla **siempre** coincida con los datos. Cambias el dato → Angular actualiza la pantalla solo.

Esa es la idea más importante de toda la asignatura:

> **En Angular, la pantalla es un reflejo de tus datos.** Tú cambias los datos (en el `.ts`) y Angular cambia la pantalla (el `.html`). Nunca tocas la pantalla a mano.

A esto se le llama programación **declarativa** (dices *qué* quieres ver) frente a **imperativa** (dices *paso a paso cómo* cambiarlo, como en el ejemplo sin Angular).

1.3 Qué te da Angular "de serie"

Pieza	Para qué	Sección
Componentes	Dividir la app en trozos (páginas, menús, tarjetas…)	5.1
Data binding	Conectar los datos del `.ts` con lo que se ve en el `.html`	5.3
Directivas (`@for`, `@if`…)	Repetir y mostrar/ocultar trozos de HTML	5.4
Router	Navegar entre pantallas según la URL	5.8
Servicios + inyección de dependencias	Lógica y datos compartidos entre pantallas	5.9
HttpClient	Pedir datos a un servidor (API)	15
CLI (`ng`, `ionic generate`…)	Crear proyectos y ficheros con comandos	2.5

---

2. Las herramientas: Node, npm, Ionic CLI, la terminal

2.1 Node.js

Node.js permite ejecutar JavaScript **fuera del navegador**, en tu ordenador. Lo necesitamos porque las herramientas (Angular CLI, Ionic CLI, el compilador de TypeScript...) son programas escritos en JavaScript.

```
node -v      # muestra la versión de Node (tú tienes la 24)
```

2.2 npm (Node Package Manager)

npm es el **gestor de paquetes** de Node. Un *paquete* es una librería que ha hecho otra persona (Angular, Ionic, Capacitor...). npm las descarga de internet.

Comparación: es como **Maven** en Java (el `pom.xml`), pero para JavaScript.

Comando	Qué hace
`npm install`	Lee `package.json` y descarga **todas** las dependencias a la carpeta `node_modules/`.
`npm install paquete`	Descarga un paquete nuevo y lo apunta en `package.json`.
`npm install -D paquete`	Igual, pero como **devDependency** (solo para desarrollar).
`npm install -g paquete`	Lo instala **globalmente** en tu PC (para usarlo como comando en cualquier carpeta, p. ej. `ionic`).
`npm run build`	Ejecuta el script `build` que está definido en `package.json`.
`npx comando`	Ejecuta un programa que está dentro de `node_modules` sin instalarlo globalmente (p. ej. `npx cap sync`).

2.3 `package.json`, `package-lock.json` y `node_modules`

`package.json` es la "ficha" del proyecto. El de tu Corporate App (resumido):

```
{
  "name": "corporateApp",
  "scripts": {
    "start": "ng serve",
```

```
    "build": "ng build"
  },
  "dependencies": {
    "@angular/core": "22.1.7",
    "@ionic/angular": "^9.0.0",
    "@capacitor/core": "^8.5.2"
  },
  "devDependencies": {
    "@angular/cli": "22.1.8",
    "typescript": "~6.0.0"
  },
  "engines": { "node": "24.x" }
}
```

Parte	Qué significa
`"name"`	Nombre del proyecto.
`"scripts"`	Atajos. `npm run build` ejecuta `ng build`.
`"dependencies"`	Librerías que la app **necesita para funcionar** (acaban dentro de la app final). Ej.: Angular, Ionic.
`"devDependencies"`	Librerías que solo se usan **mientras programas o compilas** (no van en la app final). Ej.: el compilador de TypeScript, la CLI de Angular.
`"engines"`	Qué versión de Node necesita el proyecto. Lo añadimos para que Vercel use Node 24.
`^9.0.0`	"Cualquier 9.x.x, desde la 9.0.0".
`~6.0.0`	"Cualquier 6.0.x" (más estricto).
`22.1.7`	Exactamente esa versión.

- **`node_modules/`**: la carpeta donde npm mete las librerías descargadas. Pesa cientos de MB. **Nunca se sube a GitHub** (está en `.gitignore`) porque se regenera con `npm install`.
- **`package-lock.json`**: apunta la versión **exacta** que se instaló de cada librería, para que en otro ordenador (o en Vercel) se instale lo mismo. No se toca a mano.

> Error típico que ya te pasó: lanzar `npm install` en una carpeta **sin** `package.json`. Siempre hay que estar **dentro** de la carpeta del proyecto (`corporateApp`).

2.4 La terminal (PowerShell)

Comando	Qué hace
`cd carpeta`	Entrar en una carpeta. Si el nombre tiene espacios: `cd "C:\DAM 2\Desarrollo de Interfaces"`.

Comando	Qué hace
`cd ..`	Subir una carpeta.
`ls` (o `dir`)	Ver qué hay en la carpeta actual.
`Ctrl + C`	Parar el programa que está corriendo (por ejemplo `ionic serve`).
Flecha ↑	Repetir comandos anteriores.

En VS Code la terminal se abre con **Ctrl + ñ** y se abre en la carpeta que tengas abierta.

2.5 Ionic CLI y Angular CLI

Una **CLI** (*Command Line Interface*) es un programa que se usa escribiendo comandos.

Comando	Qué hace
`ionic start nombre blank --type=angular`	Crea un proyecto nuevo con la plantilla vacía.
`ionic serve`	Compila y arranca un **servidor de desarrollo** en `http://localhost:8100`.
`ionic generate page pages/home`	Crea una página (carpeta con `.ts`, `.html`, `.scss`).
`ionic generate service services/products`	Crea un servicio.
`ionic generate component components/galeria`	Crea un componente.
`ionic build`	Compila la app para producción (carpeta `www/`).
`ionic info`	Muestra las versiones de todo.

- **`localhost:8100`**: `localhost` significa "este mismo ordenador"; `8100` es el **puerto** (como el número de puerta por el que entra el navegador). La app solo la ves tú.
- **Live reload**: mientras `ionic serve` está corriendo, cada vez que guardas un fichero (**Ctrl + S**) la página del navegador se recarga sola con los cambios.

---

3. JavaScript y TypeScript: lo mínimo para leer el código

Tú sabes Java, así que vas a ver que se parece mucho. Te pongo cada cosa con su equivalente en Java.

3.1 Variables

```
let edad = 20;          // variable que puede cambiar     (Java: int edad =
  20;)
const nombre = 'Ana';   // constante, no se puede reasignar (Java: final
  String nombre = "Ana";)
let precio: number = 9.99;   // TypeScript: con tipo explícito
```

- En TypeScript el tipo va **después** del nombre, con dos puntos: `nombre: tipo`.
- Tipos básicos: `number` (todos los números, enteros y decimales), `string` (texto), `boolean` (`true` / `false`), `any` (cualquier cosa: evitarlo).
- Los textos pueden ir con `'comillas simples'`, `"dobles"` o `` `backticks` ``. Los backticks permiten meter variables dentro: `` `Hola ${nombre}` `` → `Hola Ana`.

3.2 Arrays y objetos

```
const numeros: number[] = [1, 2, 3];        // array de números  (Java:
  int[] / List<Integer>)

const producto = {                           // objeto "literal": pares
  clave: valor
  id: 1,
  nombre: 'Portátil Dell',
  precio: 1200
};
console.log(producto.nombre);                // 'Portátil Dell'
```

En JavaScript **no hace falta una clase** para crear un objeto: se escribe directamente entre llaves `{ }`. Esto es muy distinto de Java.

3.3 JSON

JSON (*JavaScript Object Notation*) es un formato de **texto** para guardar o enviar datos. Es casi igual que un objeto JavaScript, pero las claves van **siempre entre comillas dobles**:

```
[
  { "id": 1, "nombre": "Portátil Dell", "precio": 1200 },
```

```
  { "id": 2, "nombre": "Monitor LG", "precio": 299 }
]
```

- `[ ]` = una lista (array). `{ }` = un objeto.
- `JSON.parse(texto)` convierte texto JSON → objeto. `JSON.stringify(objeto)` hace lo contrario.

3.4 Funciones y funciones flecha

```
// Función normal
function sumar(a: number, b: number): number {
  return a + b;
}

// Función flecha (arrow function): lo mismo, más corto
const sumar2 = (a: number, b: number): number => a + b;

// Con un solo parámetro se pueden quitar los paréntesis
const doble = x => x * 2;
```

La flecha `=>` se lee "devuelve". Es como las **lambdas** de Java (`(a, b) -> a + b`). Las vas a ver muchísimo: `.then(respuesta => respuesta.json())`, `.filter(u => u.active)`...

3.5 Métodos de arrays más usados

```
const usuarios = [
  { name: 'Ana', active: true },
  { name: 'Luis', active: false }
];

usuarios.filter(u => u.active);     // [{ name: 'Ana', active: true }]  → se
  queda con los que cumplen
usuarios.map(u => u.name);          // ['Ana', 'Luis']                  →
  transforma cada elemento
usuarios.length;                    // 2
```

Igual que los Streams de Java (`filter`, `map`).

3.6 Clases

```
export class Persona {
  nombre: string;                       // propiedad (atributo)
  private edad = 0;                     // privada, con valor inicial

  constructor(nombre: string) {         // constructor (en Java se llamaría
  igual que la clase)
    this.nombre = nombre;
  }

  saludar(): string {                   // método
    return `Hola, soy ${this.nombre}`;
```

```
  }
}
```

Atajo muy usado en Angular: poner `private` (o `public`) **en el parámetro del constructor** crea la propiedad automáticamente:

```
constructor(private servicio: ProductsService) {}
// equivale a:
// private servicio: ProductsService;
// constructor(servicio: ProductsService) { this.servicio = servicio; }
```

3.7 Interfaces

```
export interface Product {
  id: number;
  nombre: string;
  precio: number;
}
```

Una **interface** en TypeScript describe **la forma** que debe tener un objeto (qué campos y de qué tipo). No tiene código ni se puede instanciar con `new`. Sirve para que el editor te avise si te equivocas: si escribes `producto.nombr`, sale en rojo.

> Diferencia con Java: en Java una interface define **métodos** que una clase implementa. En TypeScript se usa sobre todo para definir **la forma de los datos** (como un "molde").

3.8 `import` / `export`

Cada fichero `.ts` es un **módulo**. Lo que quieras usar desde otro fichero tiene que llevar `export`, y el otro fichero lo trae con `import`:

```
// models/product.interface.ts
export interface Product { ... }

// services/products.service.ts
import { Product } from '../models/product.interface';
```

- `'../'` = subir una carpeta. `'./'` = la carpeta actual.
- Si la ruta **no** empieza por `.` (`'@angular/core'`), viene de `node_modules` (una librería).
- No se pone la extensión `.ts`.

3.9 Decoradores `@`

```
@Component({ ... })
export class HomePage { }
```

Un **decorador** es una etiqueta que se pone encima de una clase para darle un "superpoder" o información extra. Como las anotaciones de Java (`@Override`, `@Test`). En Angular los principales son `@Component` (esto es un componente) e `@Injectable` (esto es un servicio).

3.10 Asincronía: `Promise`, `async` y `await`

Algunas operaciones **tardan**: pedir datos a un servidor, leer un fichero, el GPS... JavaScript no se queda bloqueado esperando: sigue ejecutando y "ya avisará" cuando llegue el resultado.

- Una **Promise** (promesa) es un objeto que representa "un valor que llegará más tarde". Puede acabar **bien** (*resolved*) o **mal** (*rejected*).
- `async` delante de una función → esa función devuelve siempre una `Promise`.
- `await` delante de una promesa → "espera aquí a que llegue el resultado" (solo se puede usar dentro de una función `async`).

```
async function cargar() {
  const respuesta = await fetch('assets/data/products.json');  // espera la
   respuesta
  const datos = await respuesta.json();                        // espera a
   convertirla
  return datos;
}
```

`Promise<Product[]>` como tipo significa "una promesa que, cuando termine, dará un array de `Product`".

---

4. Práctica: paradigmas asíncronos de JavaScript (`paradigmas-js`)

📁 `01-Introduccion-Ionic/paradigmas-js/` — `index.html` + `codigo.js`

Objetivo: pedir la lista de usuarios a una API pública (`https://jsonplaceholder.typicode.com/users`) de **tres formas**

distintas y ver que hacen lo mismo. Para verlo: abrir `index.html` en el navegador → **F12** → pestaña **Console**.

`index.html`

```
<!DOCTYPE html>                          <!-- "esto es HTML5" -->
<html lang="es">                         <!-- empieza la página; idioma español
  -->
<head>                                   <!-- cabecera: info que no se ve -->
    <meta charset="UTF-8">               <!-- codificación: para que salgan
  bien tildes y ñ -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <!-- se adapta al móvil -->
    <title>Paradigmas de JavaScript</title>   <!-- texto de la pestaña -->
</head>
<body>                                   <!-- lo que se ve -->
    <h2>Paradigmas de JavaScript</h2> <!-- título de nivel 2 -->
    <p>Abre la consola ... </p>          <!-- párrafo -->
    <script src="codigo.js"></script> <!-- carga y ejecuta el fichero
  JavaScript -->
</body>
</html>
```

Caso 1 — `fetch` con `.then()`

```
function casoFetch() {
  console.log("--- CASO 1: fetch con .then() ---");     // escribe en la
   consola (como System.out.println)
  fetch("https://jsonplaceholder.typicode.com/users")    // 1) pide la URL →
   devuelve una Promise
    .then((respuesta) => respuesta.json())               // 2) cuando
   llegue: convierte el cuerpo a JSON (otra Promise)
    .then((datos) => console.log("[fetch .then] JSON recibido:", datos))  //
   3) cuando esté: lo muestra
    .catch((error) => console.error("[fetch .then] Error:", error));     //
   si algo falla en cualquier paso
}
```

- `fetch(url)` es la función del navegador para hacer peticiones HTTP.
- `.then(función)` = "cuando la promesa termine bien, ejecuta esta función con el resultado".
- Encadenar `.then().then()` = hacer pasos uno detrás de otro.
- `.catch(función)` = "si falla, ejecuta esto".

Caso 2 — `async/await`

```
async function casoAsyncAwait() {
  try {
    const respuesta = await
   fetch("https://jsonplaceholder.typicode.com/users");
    const datos = await respuesta.json();
    console.log("[async/await] JSON recibido:", datos);
```

```
  } catch (error) {
    console.error("[async/await] Error:", error);
  }
}
```

Hace **exactamente lo mismo** que el caso 1, pero se lee como código normal de arriba abajo. `try/catch` funciona igual que en Java. **Esta es la forma que usamos en Angular.**

Caso 3 — callback clásico con `XMLHttpRequest`

```
function casoCallback() {
  const xhr = new XMLHttpRequest();                    // objeto antiguo
   para peticiones HTTP
  xhr.open("GET", "https://jsonplaceholder...", true); // prepara: método
   GET, URL, true = asíncrono
  xhr.onreadystatechange = function () {               // CALLBACK: función
   que el navegador llamará él solo
    if (xhr.readyState === 4 && xhr.status === 200) {  // 4 = terminado, 200
  = OK
      const datos = JSON.parse(xhr.responseText);      // el texto recibido
  → objeto
      console.log("[callback clásico] JSON recibido:", datos);
    }
  };
  xhr.send();                                          // ahora sí, envía la
   petición
}
```

- Un **callback** es una función que le pasas a otro para que **él la llame cuando toque**.
- `===` compara valor **y** tipo (en JavaScript siempre se usa `===`, no `==`).
- Así se hacía antes de que existieran las promesas. Si encadenas muchas peticiones así, las funciones quedan unas dentro de otras ("callback hell"); por eso aparecieron `.then()` y después `async/await`.

Al final

```
casoFetch();
casoAsyncAwait();
casoCallback();
```

Se lanzan los tres seguidos. **Pregunta de examen:** ¿en qué orden salen los resultados en la consola? **No se sabe**: depende de cuál responda antes el servidor. Los tres `console.log` de "--- CASO ---" sí salen primero y en orden, porque esos no esperan nada.

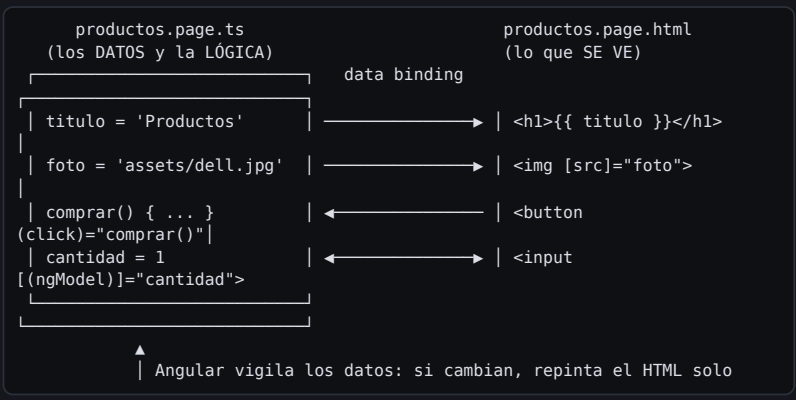
---

5. Angular: las piezas

> Antes de esta sección lee la 1.1 y 1.2: si no tienes claro qué es un framework, esto no va a encajar.

5.0 La idea central en un dibujo

Todo lo de Angular gira alrededor de esto:



- A la izquierda, una **clase normal de TypeScript** con variables y métodos (como una clase Java).
- A la derecha, **HTML normal** con unos símbolos especiales (`{{ }}`, `[ ]`, `( )`).
- Esos símbolos son el **data binding**: los "cables" que conectan las variables de la izquierda con la pantalla de la derecha. Angular es quien mantiene esos cables funcionando.

5.1 Componente

Un **componente** es **un trozo de pantalla con su propia lógica**. Puede ser una página entera (Productos) o un trozo pequeño (una tarjeta de producto, un menú, una galería).

Analogía Unity: un componente es como un **prefab** con su script: tiene su aspecto (el HTML, como el modelo 3D) y su comportamiento (el `.ts`, como el script C#). Y igual que en Unity metes prefabs dentro de otros, en Angular metes componentes dentro de otros: la app entera es un **árbol de componentes**.

```
AppComponent                (raíz: siempre está)
└── <ion-router-outlet>     (hueco donde va la página actual)
    └── ProductosPage       (la página de la ruta /productos)
        ├── <ion-header>    (componente de Ionic)
        └── <ion-grid>      (componente de Ionic)
```

Cada componente son **3 ficheros** con el mismo nombre:

Fichero	Qué contiene	Analogía
`productos.page.ts`	La **clase**: datos (variables) y comportamiento (métodos).	El script / el cerebro
`productos.page.html`	La **plantilla** (*template*): qué se dibuja.	El aspecto / el cuerpo
`productos.page.scss`	Los **estilos**: colores, tamaños... Solo afectan a este componente.	La ropa

El `.ts` es lo que "convierte" una clase normal en un componente, gracias al decorador `@Component`:

```ts
import { Component } from '@angular/core';
import { IonHeader, IonToolbar, IonTitle, IonContent } from
  '@ionic/angular';

@Component({                              // "Angular: esta clase es un
  componente"
  selector: 'app-home',                   // 1
  templateUrl: './home.page.html',        // 2
  styleUrls: ['./home.page.scss'],        // 3
  imports: [IonHeader, IonToolbar, IonTitle, IonContent],   // 4
})
export class HomePage {                   // 5
  titulo = 'Corporate App';               // 6
}
```

1. `selector`: el nombre de **etiqueta HTML** que tendrá este componente. Si en otro HTML escribes `<app-home></app-home>`, ahí se dibuja este componente. (Las páginas casi nunca se usan así porque las pone el router, pero el selector tiene que existir).
2. `templateUrl`: dónde está su HTML.
3. `styleUrls`: dónde están sus estilos (es una lista `[ ]` porque podrían ser varios).
4. `imports`: qué **otros componentes** usa su HTML (ver 5.2).
5. `export class HomePage`: una clase normal. `export` para que otros ficheros (las rutas) puedan usarla.

6. `titulo = 'Corporate App'`: una **propiedad** (variable de la clase). Todo lo que pongas aquí lo puede usar el HTML.

Qué pasa cuando abres `/home`: Angular ve en las rutas que `/home` es `HomePage` → crea un objeto `new HomePage()` (tú nunca escribes ese `new`; lo hace Angular, inversión de control) → lee su HTML → sustituye los `{{ }}` y demás por los valores reales → lo pinta en el hueco del `router-outlet`.

> **"Page" o "component"?** Para Angular son lo mismo. Ionic llama *page* a los componentes que son una pantalla completa (con ruta) y *component* a los trozos que van dentro de una página.

5.2 Standalone

Antes (Angular hasta la versión 14 más o menos), los componentes no se podían usar solos: había que registrarlos en un fichero central llamado **módulo** (`app.module.ts`, con `@NgModule`), algo así como una "lista de inscritos". Era lioso: para usar un botón en una página había que importarlo en el módulo, no en la página.

Ahora cada componente es **standalone** (*independiente*): **declara él mismo** en su `imports: [...]` todo lo que usa en su HTML. No hay módulos.

Regla práctica (la que más errores te va a quitar):

> Cada etiqueta especial que escribas en el HTML tiene que estar en el `imports` del `.ts`.

Si en el `.html` usas...	En el `.ts` necesitas importar...	Desde
`<ion-button>`	`IonButton`	`'@ionic/angular'`
`<ion-grid>`, `<ion-row>`, `<ion-col>`	`IonGrid`, `IonRow`, `IonCol`	`'@ionic/angular'`
`routerLink="..."`	`RouterLink`	`'@angular/router'`

Si en el `.html` usas...	En el `.ts` necesitas importar...	Desde
`*ngFor` / `*ngIf`	`NgFor` / `NgIf` (o `CommonModule`)	`'@angular/common'`
`[(ngModel)]`	`FormsModule`	`'@angular/forms'`
`| currency`	`CurrencyPipe`	`'@angular/common'`
`@for` / `@if`	**nada** (vienen de serie)	—

Si te olvidas, sale un error del tipo *"'ion-button' is not a known element"* o *"Can't bind to 'ngForOf'"*. Fíjate que el nombre siempre es el de la etiqueta en **PascalCase**: `ion-card-title` → `IonCardTitle`.

Para importarlo se hace en **dos sitios** del mismo fichero:

```
import { IonButton } from '@ionic/angular';   // ① arriba: traer la clase
  al fichero (TypeScript)

@Component({
  imports: [IonButton],                       // ② en el decorador:
  decirle a Angular que el HTML la usa
})
```

5.3 Data binding: cómo se conecta el `.ts` con el `.html`

El problema

El `.ts` y el `.html` son **dos ficheros distintos**. Si en el `.ts` tienes `titulo = 'Productos'`, ¿cómo sabe el HTML que tiene que escribir "Productos"? Y si en el HTML hay un botón, ¿cómo llama a un método del `.ts`?

Data binding (en español, "enlace de datos") es la respuesta: unos **símbolos especiales** que pones en el HTML para conectarlo con la clase. "Binding" = atadura, enlace, cable.

Hay **4 tipos**, y se distinguen **solo por los símbolos**:

```
 {{ dato }}           interpolación       .ts ──► .html   (escribir un
texto)
 [atributo]="dato"    property binding    .ts ──► .html   (rellenar un
atributo)
 (evento)="metodo()"  event binding       .ts ◄── .html   (avisar de que ha
pasado algo)
```

```
 [(ngModel)]="dato"   two-way binding     .ts ◄► .html   (las dos cosas a
la vez)
```

Vamos a verlos todos con **el mismo ejemplo**, una mini tienda:

```ts
// tienda.page.ts
export class TiendaPage {
  producto = 'Portátil Dell';
  precio = 1200;
  foto = 'assets/images/dell.jpg';
  agotado = false;
  cantidad = 1;

  comprar() {
    console.log('Has comprado ' + this.cantidad + ' ' + this.producto);
  }
}
```

1) Interpolación: `{{ }}`

Qué hace: escribe el valor de una variable **como texto** dentro del HTML.

```html
<h1>{{ producto }}</h1>              <!-- se ve: Portátil Dell -->
<p>Precio: {{ precio }} €</p>        <!-- se ve: Precio: 1200 € -->
```

Cómo leerlo: "aquí dentro va el valor de `producto`". Las dobles llaves son como un hueco que Angular rellena. Es el equivalente a hacer en Java:

```
System.out.println("Precio: " + precio + " €");
```

Dentro de `{{ }}` puedes poner expresiones sencillas, no solo variables:

```html
<p>{{ precio * 2 }}</p>                       <!-- 2400 -->
<p>{{ producto.toUpperCase() }}</p>           <!-- PORTÁTIL DELL -->
<p>{{ agotado ? 'Agotado' : 'Disponible' }}</p>   <!-- Disponible (operador
  ternario, como en Java) -->
```

Lo que **no** puedes poner: sentencias como `if`, `for`, `let x = 3` o asignaciones (`precio = 5`). Solo cosas que **den un valor**.

Y si la variable cambia (por ejemplo `this.precio = 999` desde un método), Angular vuelve a escribir el texto solo. Eso es lo que no hacía el JavaScript sin framework.

2) Property binding: `[atributo]="variable"`

Primero, ¿qué es un **atributo**? Las etiquetas HTML llevan **atributos**, que son ajustes con la forma `nombre="valor"`:

```
<img src="assets/images/dell.jpg" width="80">
<!--  ↑ atributo src     ↑ atributo width -->
<button disabled>Comprar</button>
<!--     ↑ atributo disabled (el botón no se puede pulsar) -->
```

El problema: si escribes una variable dentro de un atributo normal, HTML no sabe que es una variable; la toma como **texto literal**:

```
<img src="foto">          <!-- ❌ busca un fichero que se llama
  literalmente "foto" -->
```

La solución: poner el atributo **entre corchetes**. Los corchetes significan *"lo que hay entre comillas NO es un texto: es código TypeScript, evalúalo"*:

```
<img [src]="foto">        <!-- ✅ src = el VALOR de la variable foto =
  'assets/images/dell.jpg' -->
```

Compara:

Escribes	Angular entiende	Resultado
`src="foto"`	El texto `"foto"`	Imagen rota
`[src]="foto"`	El valor de la variable `foto`	`assets/images/dell.jpg` ✅
`src="{{ foto }}"`	Interpolación dentro del atributo (también vale para textos)	`assets/images/dell.jpg` ✅

Más ejemplos con nuestra tienda:

```
<button [disabled]="agotado">Comprar</button>
<!-- si agotado es true, el botón se desactiva; si cambia a false, se activa
  solo -->

<ion-button [color]="agotado ? 'medium' : 'success'">Comprar</ion-button>
<!-- botón gris si está agotado, verde si no -->

<p [style.color]="precio > 1000 ? 'red' : 'black'">{{ precio }} €</p>
<!-- texto rojo si es caro -->
```

Truco para recordarlo: los corchetes `[ ]` parecen una **caja donde METES** un valor: el dato **entra** en el HTML.

3) Event binding: `(evento)="metodo()"`

Hasta ahora los datos iban del `.ts` al `.html`. Ahora al revés: el usuario **hace algo** en la pantalla (pulsar, escribir...) y queremos que se ejecute código del `.ts`.

```
<ion-button (click)="comprar()">Comprar</ion-button>
```

Cómo leerlo: "cuando ocurra el evento `click` en este botón, ejecuta el método `comprar()` de la clase". Los paréntesis rodean el **nombre del evento**.

Sin Angular habría que escribir `document.getElementById('boton').addEventListener('click', comprar)`. En Java Swing sería un `boton.addActionListener(e -> comprar())`. Es la misma idea, pero escrita directamente en el HTML.

Eventos habituales:

Evento	Cuándo pasa
`(click)`	Al pulsar
`(input)`	Cada vez que se escribe una letra en un campo
`(change)`	Cuando cambia el valor y se sale del campo
`(submit)`	Al enviar un formulario
`(ionChange)`	Evento propio de componentes Ionic (selects, toggles...)

Si necesitas los datos del evento (qué tecla, qué se ha escrito...), Angular te los da en la variable especial `$event`: `(input)="buscar($event)"`.

Truco para recordarlo: los paréntesis `( )` son como unas **orejas que escuchan**: el HTML **avisa** al código.

4) Two-way binding: `[(ngModel)]="variable"`

Es la combinación de los dos anteriores: `[ ]` + `( )` = `[( )]`. Sirve para campos de formulario donde el dato tiene que ir **en los dos sentidos**:

```
<ion-input [(ngModel)]="cantidad" type="number"></ion-input>
<p>Vas a comprar {{ cantidad }} unidades</p>
```

Qué pasa:

- `.ts → .html`: al abrir la página, el campo muestra `1` (el valor inicial de `cantidad`).
- `.html → .ts`: el usuario escribe `3` → la variable `cantidad` pasa a valer `3` **sola**.
- Como `cantidad` ha cambiado, el `<p>` de debajo se actualiza a "Vas a comprar 3 unidades" **mientras escribes**.

Por dentro, `[(ngModel)]="cantidad"` es un atajo de: `[ngModel]="cantidad" (ngModelChange)="cantidad = $event"` (meter el valor + escuchar cambios).

- Necesita importar `FormsModule` (de `'@angular/forms'`) en el componente.
- La forma `[( )]` se llama **"banana in a box"** 🍌📦: el plátano `( )` dentro de la caja `[ ]`. Si lo escribes al revés, `([ngModel])`, no funciona.

Resumen para memorizar

Símbolo	Nombre	Dirección	Para qué	Ejemplo
`{{ }}`	Interpolación	`.ts → .html`	Escribir un texto	`{{ producto }}`
`[ ]`	Property binding	`.ts → .html`	Rellenar un atributo	`[src]="foto"`
`( )`	Event binding	`.html → .ts`	Reaccionar a algo del usuario	`(click)="comprar()"`
`[( )]`	Two-way binding	`.ts ↔ .html`	Campos de formulario	`[(ngModel)]="cantidad"`

> Regla: **corchetes = entra un dato en el HTML. Paréntesis = sale un aviso del HTML.**

Practica leyendo tu propio código

En `02-CorporateApp/corporateApp/src/app/pages/productos/productos.page.html`, encuentra:

1. Una **interpolación** → `{{ product.nombre }}`, `{{ product.precio }} €`...
2. Un **property binding** → `<img [src]="product.foto">`
3. ¿Hay algún **event binding**? → No en esa página. En la actividad de dummyjson sí: `<ion-button (click)="loadProducts()">Reintentar</ion-button>`.

5.4 Directivas de control: repetir y condicionar

Las **directivas** son instrucciones en el HTML para **repetir** o **mostrar/ocultar** trozos. Son el `for` y el `if` de la plantilla.

Sintaxis nueva (Angular 17+, la que recomienda tu profe; no hay que importar nada):

```
@for (user of users; track user.id) {     <!-- for-each: un <li> por cada
   usuario -->
  <li>{{ user.name }}</li>
}

@if (loading) {                           <!-- if: solo se pinta si loading
   es true -->
  <p>Cargando...</p>
} @else {
  <p>Listo</p>
}
```

- `@for (user of users; ...)` = Java `for (User user : users)`. Dentro de las llaves, `user` es el elemento de esa vuelta.
- `track user.id` le dice a Angular qué identifica a cada elemento, para que, si cambia la lista, solo redibuje las filas que han cambiado. Es **obligatorio** en `@for`.
- Extra: `@for (...) { ... } @empty { <p>No hay nada</p> }` → lo que se muestra si la lista está vacía.

Sintaxis clásica (la verás en PDFs antiguos y en tu `usersApp`):

```
<li *ngFor="let user of users">{{ user.name }}</li>
<p *ngIf="loading">Cargando...</p>
```

El `*` delante indica que la directiva **cambia la estructura** del HTML (añade o quita elementos). Hay que importar `NgFor` / `NgIf` (o `CommonModule`). Hacen lo mismo que `@for` / `@if`.

5.5 Pipes `|`

Un **pipe** ("tubería") transforma un valor **solo para mostrarlo**, sin cambiar la variable:

```
{{ precio | currency:'EUR' }}       <!-- €1,200.00  (formato inglés, el que
  viene por defecto) -->
{{ fecha | date:'short' }}          <!-- 10/2/26, 9:31 AM -->
{{ nombre | uppercase }}            <!-- ANA -->
```

Se lee "coge `precio` y pásalo por la tubería `currency`". Lo que va después de `:` son opciones. Hay que importarlos en el componente (`CurrencyPipe`, `DatePipe`, `UpperCasePipe`... de `'@angular/common'`). En tu Corporate App el precio se pone a mano: `{{ product.precio }} €`.

5.6 Signals (y por qué los usamos)

Volvamos a la idea central: **cuando cambian los datos, Angular repinta la pantalla**. Pero... ¿cómo se **entera** Angular de que un dato ha cambiado?

- **Antes** usaba una librería llamada **zone.js** que espiaba todo lo que pasaba en la página (clics, temporizadores, peticiones...) y, después de cada cosa, revisaba todas las variables por si alguna había cambiado. Funcionaba "por arte de magia", pero era lento.
- **Angular 22 ya no usa zone.js** (se llama *zoneless*). Ahora hay que **avisarle**. Si cambias una variable normal dentro de algo asíncrono (después de un `await` o dentro de un `subscribe`), **Angular no se entera** y la pantalla se queda como estaba.

```
products: Product[] = [];
async ngOnInit() {
  this.products = await this.servicio.getProducts();   // ❌ la variable
   cambia, la pantalla NO
}
```

Este fue exactamente el bug de `usersApp` ("Cargando..." para siempre).

La solución son los **signals** ("señales"). Un signal es una **caja que guarda un valor y que avisa a Angular cada vez que cambia lo que hay dentro**:

```
import { signal } from '@angular/core';

products = signal<Product[]>([]);      // CREAR: caja de tipo Product[],
   empieza con [] dentro

this.products.set(nuevaLista);         // ESCRIBIR: cambia el contenido Y
   avisa a Angular → repinta
this.products();                       // LEER: se "abre la caja" llamándola
   como una función
```

En el HTML también se lee con paréntesis:

```
@for (product of products(); track product.id) { ... }
<p>Hay {{ products().length }} productos</p>
```

Analogía: una variable normal es una pizarra donde cambias lo escrito sin que nadie se entere. Un signal es un grupo de WhatsApp: cada vez que cambias algo, **les llega una notificación** a todos los que estaban mirando.

> **Regla práctica para tu Angular 22:** todo dato que se muestre en el HTML y que cambie **después** de que la página se haya abierto (porque llega de un servicio, de una API, de un temporizador...) → **signal**.

5.7 Ciclo de vida: `ngOnInit`

Como Angular es un framework, **él** crea y destruye los componentes, y te avisa en ciertos momentos llamando a métodos con nombres especiales (como `Start()` en Unity). El más usado:

```
export class ProductosPage implements OnInit {   // "implements OnInit" =
   prometo tener ngOnInit
  ngOnInit() {
    // Angular lo llama UNA vez, justo después de crear el componente
    // → es el sitio para cargar datos
  }
}
```

Unity	Angular
`Awake()`	`constructor()`
`Start()`	`ngOnInit()`
`OnDestroy()`	`ngOnDestroy()`

¿Por qué no cargar los datos en el constructor? Por convenio: el constructor solo **recibe** las dependencias (servicios); el trabajo de verdad va en `ngOnInit`.

5.8 Rutas (routing)

Una app tiene **varias pantallas**, pero en realidad solo hay **un** `index.html` (es una SPA, *Single Page Application*). El **router** de Angular decide **qué componente se pinta según la URL**:

```
localhost:8100/home        → HomePage
localhost:8100/productos   → ProductosPage
```

```
export const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },     //
   localhost:8100/ → /home
  { path: 'home', loadComponent: () =>
   import('./pages/home/home.page').then(m => m.HomePage) },
];
```

- `path`: lo que va en la URL después de la barra.
- `redirectTo`: redirige a otra ruta. `pathMatch: 'full'`: solo si la URL es **exactamente** esa (vacía).
- `loadComponent: () => import(...)`: **lazy loading** (carga perezosa). El código de esa página no se descarga hasta que el usuario entra en ella, así la app arranca más rápido.
  - `() => ...` es una función flecha (sección 3.4) que Angular llamará **cuando haga falta**.
  - `import('./pages/home/home.page')` carga ese fichero (devuelve una Promise).
  - `.then(m => m.HomePage)` de ese fichero (`m`) coge la clase `HomePage`.
- `<router-outlet>` / `<ion-router-outlet>` es el **hueco** del HTML donde se pinta la página que toque. Está en el componente raíz.
- `routerLink="/productos"` en un botón = "al pulsarlo, navega a `/productos`".

5.9 Servicios e inyección de dependencias (DI)

Un **servicio** es una clase **que no se ve**: se encarga de la lógica y de conseguir los datos (pedir productos, guardar mensajes, usar el GPS...). Las páginas lo usan.

¿Por qué no poner el `fetch` directamente en la página? Porque si tres páginas necesitan los productos, tendrías el mismo código tres veces, y si cambia la URL tendrías que cambiarlo en tres sitios. Con un servicio, está **en un único sitio**.

```
@Injectable({ providedIn: 'root' })     // "esto es un servicio, y hay UNO
   para toda la app"
export class ProductsService {
  async getProducts(): Promise<Product[]> { ... }
}
```

Inyección de dependencias = la página **no crea** el servicio con `new`; **lo pide**, y Angular se lo da ya creado:

```
constructor(private productService: ProductsService) {}   // forma clásica
private productService = inject(ProductsService);          // forma moderna
   (hace lo mismo)
```

Analogía: en un restaurante, el camarero (la página) no construye su propia cocina (el servicio). La cocina ya existe, y el restaurante (Angular) se la asigna. Todos los camareros usan **la misma** cocina.

Ventajas:

1. **Una sola instancia** compartida por toda la app (*singleton*) gracias a `providedIn: 'root'`.
2. **Bajo acoplamiento**: si cambias de dónde salen los datos (JSON → API real), solo tocas el servicio; las páginas ni se enteran.
3. **Fácil de probar**: en un test se le puede dar a la página un servicio "falso".

Palabras clave para el profe: **dependencia** (lo que se necesita), **proveedor/provider** (la receta para crearla: `providedIn: 'root'`), **inyector** (el que la guarda y la reparte), **inversión de control** (crear objetos deja de ser cosa del componente; otra vez la idea del framework de la sección 1.1).

5.10 `main.ts`: el arranque

```
bootstrapApplication(AppComponent, {    // "arranca la app empezando por
   AppComponent"
  providers: [ ... ]                    // configuración global: rutas,
   Ionic, HttpClient...
});
```

Orden de arranque: el navegador abre `index.html` (que tiene `<app-root>`) → se ejecuta `main.ts` → Angular crea `AppComponent` (el de selector `app-root`, que tiene el `router-outlet`) → el router mira la URL y mete la página que toca dentro del hueco.

5.11 Todo junto: qué pasa al abrir `/productos` en tu Corporate App

1. El navegador carga `index.html` y `main.ts` arranca Angular.
2. El **router** ve la URL `/productos` → descarga `productos.page.ts` (lazy loading).
3. Angular necesita crear `ProductosPage`. Ve que el constructor pide un `ProductsService` → **inyección de dependencias**: se lo da.
4. Angular crea la página y llama a `ngOnInit()` (ciclo de vida).
5. `ngOnInit` pide los productos al **servicio**, que hace el `fetch` al JSON (**asíncrono**: tarda).
6. Mientras tanto, Angular ya pinta el HTML con `products()` vacío (tabla sin filas).
7. Llegan los datos → `this.products.set(lista)` → el **signal** avisa a Angular.
8. Angular repinta: el `*ngFor` crea una fila por producto, la **interpolación** escribe nombre, precio... y el **property binding** `[src]` pone cada foto.

Si entiendes estos 8 pasos, entiendes el 80 % de lo que hemos hecho.

---

6. Práctica 1: el componente galería (data binding)

Esta tarea (21-09) la hiciste en un proyecto aparte (`myApp`), que **no está en este repositorio**. Lo que se pedía y lo que significa:

Punto de la tarea	Qué es
Versiones de Node, Ionic CLI, Angular CLI	`node -v`, `ionic -v`, `ng version`: comprobar el entorno.
`package.json`: dependencies vs devDependencies	Ver 2.3.

Punto de la tarea	Qué es
`ionic serve` y live reload	Ver 2.5.
Imagen en `src/assets`	`assets` es la carpeta de ficheros estáticos (imágenes, JSON) que se copian tal cual a la app final. Se referencian como `assets/foto1.png`.
Componente `galeria` con interpolación y `[src]`	Un componente que muestra un título con `{{ }}` y una imagen con `[src]`.

Así sería un componente galería equivalente (para que entiendas la idea):

```
ionic generate component components/galeria
```

```
// galeria.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-galeria',               // se usará como <app-galeria>
  </app-galeria>
  templateUrl: './galeria.component.html',
  styleUrls: ['./galeria.component.scss'],
})
export class GaleriaComponent {
  titulo = 'Mi galería';                  // dato para la interpolación
  imagen = 'assets/foto1.png';            // dato para el property binding
  descripcion = 'Foto guardada en src/assets';
}
```

```
<!-- galeria.component.html -->
<h2>{{ titulo }}</h2>                     <!-- interpolación: escribe "Mi
  galería" -->
<img [src]="imagen" [alt]="descripcion" width="300">   <!-- property binding
  -->
<p>{{ descripcion }}</p>
```

Y para que aparezca en una página, en su `.ts` se importa `GaleriaComponent` (en `imports`) y en su `.html` se escribe `<app-galeria></app-galeria>`.

---

7. Práctica: `usersApp` línea a línea

📁 `01-Introduccion-Ionic/usersApp/` — proyecto **Angular puro** (todavía sin componentes de Ionic). Muestra los usuarios **activos** de una lista.

Es el mismo esquema que la Corporate App, en pequeño: **modelo → servicio → página → HTML**.

`src/app/models/user.model.ts`

```
export interface User {   // molde de un usuario; export para usarlo en
   otros ficheros
  id: number;             // identificador numérico
  name: string;           // nombre
  email: string;          // correo
  active: boolean;        // ¿está activo? true / false
}
```

`src/app/services/users.service.ts`

```
import { Injectable } from '@angular/core';      // el decorador para
   servicios
import { User } from '../models/user.model';     // el molde (subimos de
   services/ a app/ y entramos en models/)

@Injectable({
  providedIn: 'root'                             // una instancia para toda
   la app
})
export class UsersService {
  private users: User[] = [                      // array de User, privado:
   solo lo usa el servicio
    { id: 1, name: 'Ana', email: 'ana@test.com', active: true },
    { id: 2, name: 'Luis', email: 'luis@test.com', active: false },
    { id: 3, name: 'Carlos', email: 'carlos@test.com', active: true }
  ];

  async getUsers(): Promise<User[]> {            // devuelve "una promesa de
   array de User"
    return new Promise(resolve => {              // creamos una promesa a
   mano...
      setTimeout(() => resolve(this.users), 500);// ...que se cumple a los
   500 ms con la lista
    });                                          // (simula lo que tardaría
   un servidor real)
  }

  async getActiveUsers(): Promise<User[]> {
    const users = await this.getUsers();         // espera la lista completa
    return users.filter(u => u.active);          // devuelve solo los que
   tienen active = true
  }
}
```

- `new Promise(resolve => ...)`: forma de crear una promesa. `resolve(valor)` = "ya está, el resultado es este".
- `setTimeout(funcion, 500)`: ejecuta la función dentro de 500 milisegundos.

src/app/pages/users/users.page.ts

```ts
import { Component, OnInit, signal } from '@angular/core';
import { CommonModule } from '@angular/common';                // trae *ngIf y *ngFor
import { UsersService } from '../../services/users.service';   // ../../ = sube dos carpetas (users/ y pages/)
import { User } from '../../models/user.model';

@Component({
  selector: 'app-users',
  standalone: true,                    // componente independiente (en Angular 22 ya es así por defecto)
  imports: [CommonModule],             // lo que usa el HTML
  templateUrl: './users.page.html'
})
export class UsersPage implements OnInit {
  users = signal<User[]>([]);          // caja con la lista; empieza vacía
  loading = signal(false);             // caja con un boolean; empieza en false

  constructor(private usersService: UsersService) {}   // INYECCIÓN DE DEPENDENCIAS

  async ngOnInit() {                   // al crearse la página...
    await this.loadUsers();            // ...carga los usuarios
  }

  async loadUsers() {
    try {
      this.loading.set(true);                                  // muestra "Cargando..."
      const data = await this.usersService.getActiveUsers();   // pide los activos y espera
      this.users.set(data);                                    // guarda → Angular repinta
    } catch (error) {
      console.error('Error:', error);                          // si falla, lo muestra en consola
    } finally {
      this.loading.set(false);                                 // pase lo que pase, quita "Cargando..."
    }
  }
}
```

`finally` se ejecuta siempre, haya error o no (igual que en Java).

src/app/pages/users/users.page.html

```html
<h1>Usuarios activos</h1>

<p *ngIf="loading()">Cargando...</p>                    <!-- solo si loading vale true -->

<ul *ngIf="!loading()">                                  <!-- ! = NOT: solo si NO está cargando -->
  <li *ngFor="let user of users()">                      <!-- un <li> por cada usuario -->
```

```
    <strong>{{ user.name }}</strong> — {{ user.email }} <!-- interpolación -
  ->
  </li>
</ul>
```

Resultado: "Ana — ana@test.com" y "Carlos — carlos@test.com" (Luis no, porque no está activo).

`src/app/app.routes.ts`

```
export const routes: Routes = [
  { path: '', redirectTo: 'users', pathMatch: 'full' },        // / → /users
  {
    path: 'users',
    loadComponent: () => import('./pages/users/users.page').then(m =>
  m.UsersPage)
  }
];
```

`src/app/app.ts` y `app.html` (el componente raíz)

```
@Component({
  imports: [RouterOutlet],          // necesita el hueco de rutas
  selector: 'app-root',             // <app-root> está en index.html
  styleUrl: './app.scss',
  templateUrl: './app.html',
})
export class App {
  protected readonly title = signal('usersApp');   // protected: solo la
  clase y su HTML; readonly: no se reasigna
}
```

```
<h2>usersApp</h2>
<router-outlet />          <!-- aquí se pinta la página de la ruta actual
  (UsersPage) -->
```

`src/main.ts` y `app.config.ts`

```
bootstrapApplication(App, appConfig)          // arranca la app con el
  componente App y esta configuración
  .catch((err) => console.error(err));        // si falla el arranque,
  muéstralo
```

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),     // captura errores globales
    provideRouter(routes)                     // activa las rutas de
  app.routes.ts
  ]
};
```

---

8. Práctica 2: `corporateApp` línea a línea

📁 `02-CorporateApp/corporateApp/` — App de una empresa con Home (menú), Productos (tabla desde JSON), Nosotros y Contacto (vacías por ahora). Publicada en https://desarrollo-interfaces-dam-2.vercel.app.

8.1 El viaje de los datos

```
products.json ──fetch──▶ ProductsService ──(inyección)──▶ ProductosPage
──signal──▶ productos.page.html
  (datos)                (los va a buscar)                (los guarda)
(los pinta)
```

8.2 Estructura

```
corporateApp/
├── package.json            ficha del proyecto (ver 2.3)
├── angular.json            configuración de Angular: dónde compila (www/),
qué estilos carga...
├── capacitor.config.ts     configuración de Capacitor (nombre de la app,
carpeta www)
├── ionic.config.json       configuración de Ionic CLI
├── vercel.json             regla para Vercel (ver 8.12)
└── src/
    ├── index.html          la única página HTML real; contiene <app-root>
    ├── main.ts             arranque
    ├── global.scss         estilos para toda la app (importa los CSS de
Ionic)
    ├── theme/variables.scss colores del tema
    ├── assets/             ficheros estáticos: data/products.json,
images/*.jpg
    └── app/
        ├── app.component.ts / .html   componente raíz
        ├── app.routes.ts              rutas
        ├── models/product.interface.ts
        ├── services/products.service.ts
        └── pages/home, productos, nosotros, contacto
```

8.3 `src/main.ts`

```ts
import { bootstrapApplication } from '@angular/platform-browser';
import { RouteReuseStrategy, provideRouter, withComponentInputBinding,
  withPreloading, PreloadAllModules } from '@angular/router';
import { IonicRouteStrategy, provideIonicAngular } from '@ionic/angular';
import { routes } from './app/app.routes';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, {
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
    provideIonicAngular(),
    provideRouter(routes, withPreloading(PreloadAllModules),
  withComponentInputBinding()),
```

```
  ],
});
```

Línea	Qué hace
`bootstrapApplication(AppComponent, {...})`	Arranca la app desde `AppComponent`.
`providers: [...]`	Configura el **inyector raíz** (lo que Angular puede "dar" a quien lo pida).
`{ provide: RouteReuseStrategy, useClass: IonicRouteStrategy }`	DI: "cuando el router pida una `RouteReuseStrategy`, dale la de Ionic". Hace que la navegación tenga animaciones y pila de páginas tipo móvil.
`provideIonicAngular()`	Activa Ionic.
`provideRouter(routes, ...)`	Activa las rutas. `withPreloading(PreloadAllModules)`: cuando la app ya ha cargado, descarga en segundo plano el resto de páginas para que luego vayan rápido. `withComponentInputBinding()`: permite pasar datos de la URL a la página (no lo usamos todavía).

8.4 `src/app/app.component.ts` y `.html`

```
@Component({
  selector: 'app-root',
  templateUrl: 'app.component.html',
  imports: [IonApp, IonRouterOutlet],
})
export class AppComponent {
  constructor() {}
}
```

```
<ion-app>                          <!-- contenedor raíz obligatorio de Ionic
  -->
  <ion-router-outlet></ion-router-outlet>   <!-- hueco donde va la página de
  la ruta -->
</ion-app>
```

8.5 `src/app/app.routes.ts`

```
export const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: 'home',      loadComponent: () =>
   import('./pages/home/home.page').then((m) => m.HomePage) },
  { path: 'productos', loadComponent: () =>
   import('./pages/productos/productos.page').then((m) => m.ProductosPage) },
  { path: 'nosotros',  loadComponent: () =>
   import('./pages/nosotros/nosotros.page').then((m) => m.NosotrosPage) },
```

```
  { path: 'contacto',  loadComponent: () =>
   import('./pages/contacto/contacto.page').then((m) => m.ContactoPage) },
];
```

`Routes` es un array de objetos; cada objeto es una ruta. Todo lo demás está explicado en 5.8. Si al compilar ves en la terminal "Lazy chunk files: home-page, productos-page...", esos son los trozos separados que genera el lazy loading.

8.6 Home: `home.page.ts`

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';            // para usar
   routerLink en el HTML
import {
  IonHeader, IonToolbar, IonTitle, IonContent, IonSegment, IonSegmentButton,
   IonLabel,
} from '@ionic/angular';                                  // cada etiqueta
   <ion-...> que usa el HTML

@Component({
  selector: 'app-home',
  templateUrl: './home.page.html',
  styleUrls: ['./home.page.scss'],
  imports: [RouterLink, IonHeader, IonToolbar, IonTitle, IonContent,
   IonSegment, IonSegmentButton, IonLabel],
})
export class HomePage {
  constructor() {}                // no necesita nada: solo muestra el menú
}
```

8.7 Home: `home.page.html`

```
<ion-header>                          <!-- cabecera fija arriba -->
  <ion-toolbar>                       <!-- la barra de la cabecera -->
    <ion-title>Corporate App</ion-title>   <!-- título de la barra -->
  </ion-toolbar>
</ion-header>

<ion-content>                         <!-- zona con scroll: el contenido de
   la página -->
  <ion-segment>                       <!-- barra de botones horizontal (el
   "menú") -->
    <ion-segment-button value="productos" routerLink="/productos">   <!--
   value: id del botón; routerLink: a dónde va -->
      <ion-label>Productos</ion-label>                                <!--
   texto del botón -->
    </ion-segment-button>
    <ion-segment-button value="nosotros" routerLink="/nosotros">
      <ion-label>Nosotros</ion-label>
    </ion-segment-button>
    <ion-segment-button value="contacto" routerLink="/contacto">
      <ion-label>Contacto</ion-label>
    </ion-segment-button>
  </ion-segment>
</ion-content>
```

Estructura **típica de cualquier página Ionic**: `ion-header > ion-toolbar > ion-title` arriba y `ion-content` con el contenido.

8.8 El modelo: `models/product.interface.ts`

```
export interface Product {
  id: number;
  nombre: string;
  vendedor: string;     // ← añadido en la ampliación de la entrega
  unidades: number;
  precio: number;
  foto: string;         // ruta de la imagen, p. ej.
  "assets/images/dell.jpg"
}
```

8.9 Los datos: `assets/data/products.json`

```
[
  { "id": 1, "nombre": "Portátil Dell", "vendedor": "Ana García",
  "unidades": 12, "precio": 1200, "foto": "assets/images/dell.jpg" },
  { "id": 2, "nombre": "Monitor LG", "vendedor": "Luis Martín", "unidades":
  8, "precio": 299, "foto": "assets/images/lg.jpg" },
  { "id": 3, "nombre": "Teclado Logitech", "vendedor": "Carlos Pérez",
  "unidades": 25, "precio": 89, "foto": "assets/images/logitech.jpg" }
]
```

Es una **base de datos falsa** ("datos fake"). Como está en `assets`, al compilar se copia tal cual y se puede pedir por URL: `localhost:8100/assets/data/products.json` (pruébalo en el navegador).

8.10 El servicio: `services/products.service.ts`

```
import { Injectable } from '@angular/core';
import { Product } from '../models/product.interface';

@Injectable({
  providedIn: 'root',                                    // servicio único
  para toda la app
})
export class ProductsService {

  async getProducts(): Promise<Product[]> {              // devolverá (más
  tarde) un array de Product
    const response = await fetch('assets/data/products.json');   // pide el
  fichero y espera
    const products = await response.json();              // convierte el
  texto JSON en array de objetos
    return products;                                     // lo devuelve a
  quien lo pidió
  }

}
```

8.11 La página Productos

productos.page.ts

```
import { Component, OnInit, signal } from '@angular/core';
import { NgFor } from '@angular/common';                    // para *ngFor
import {
  IonHeader, IonToolbar, IonTitle, IonContent, IonButtons, IonBackButton,
  IonGrid, IonRow, IonCol,
} from '@ionic/angular';
import { ProductsService } from '../../services/products.service';
import { Product } from '../../models/product.interface';

@Component({
  standalone: true,
  selector: 'app-productos',
  templateUrl: './productos.page.html',
  styleUrls: ['./productos.page.scss'],
  imports: [NgFor, IonHeader, IonToolbar, IonTitle, IonContent, IonButtons,
  IonBackButton, IonGrid, IonRow, IonCol],
})
export class ProductosPage implements OnInit {
  products = signal<Product[]>([]);                         // caja con la
  lista, empieza vacía

  constructor(private productService: ProductsService) {}  // DI: Angular
  nos da el servicio

  async ngOnInit() {                                        // al crearse la
  página:
    this.products.set(await this.productService.getProducts());   // pide
  productos, espera, guarda
  }
}
```

La línea de `ngOnInit` hace 3 cosas a la vez; desglosada sería:

```
const lista = await this.productService.getProducts();   // 1) pedir y
  esperar
this.products.set(lista);                                  // 2) guardar en
  el signal → 3) Angular repinta
```

productos.page.html

```
<ion-header>
  <ion-toolbar>
    <ion-buttons slot="start">                          <!-- grupo de
  botones a la izquierda (slot="start") -->
      <ion-back-button defaultHref="/home"></ion-back-button>   <!-- flecha
  atrás; si no hay historial, va a /home -->
    </ion-buttons>
    <ion-title>Productos</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content>
```

```
  <ion-grid>                                          <!-- rejilla
  (tabla) de Ionic -->
    <ion-row class="header">                          <!-- fila de
  cabecera; class para darle estilo -->
      <ion-col size="1">ID</ion-col>                  <!-- columna que
  ocupa 1 de 12 partes -->
      <ion-col size="2.5">Nombre</ion-col>
      <ion-col size="2.5">Vendedor</ion-col>
      <ion-col size="1.5">Uds.</ion-col>
      <ion-col size="2">Precio</ion-col>
      <ion-col size="2.5">Foto</ion-col>
    </ion-row>

    <ion-row *ngFor="let product of products()">      <!-- una fila por
  producto -->
      <ion-col size="1">{{ product.id }}</ion-col>
      <ion-col size="2.5">{{ product.nombre }}</ion-col>
      <ion-col size="2.5">{{ product.vendedor }}</ion-col>
      <ion-col size="1.5">{{ product.unidades }}</ion-col>
      <ion-col size="2">{{ product.precio }} €</ion-col>
      <ion-col size="2.5">
        <img [src]="product.foto" width="60" />       <!-- property
  binding a la ruta de la foto -->
      </ion-col>
    </ion-row>
  </ion-grid>
</ion-content>
```

- El **grid de Ionic divide el ancho en 12 partes**. `size="2.5"` = 2,5 de 12. Si sumas todas: 1 + 2,5 + 2,5 + 1,5 + 2 + 2,5 = **12**. Sin `size`, todas las columnas serían iguales.

`productos.page.scss`

```
ion-row {                          /* a todas las filas: */
  align-items: center;             /*   centrar verticalmente el contenido */
  border-bottom: 1px solid #ddd;   /*   línea gris debajo */
}
.header {                          /* a lo que tenga class="header": */
  font-weight: bold;               /*   negrita */
}
ion-col {
  overflow-wrap: anywhere;         /* si una palabra no cabe, se parte */
}
.header ion-col {                  /* columnas dentro de la cabecera: */
  font-size: 0.85rem;              /*   letra un poco más pequeña */
  overflow-wrap: normal;           /*   no partir "Vendedor" a la mitad */
}
```

En CSS: `etiqueta { }` aplica a esas etiquetas; `.clase { }` a lo que tenga ese `class`; `.a b { }` a los `b` que están **dentro** de `.a`.

8.12 Ficheros para el despliegue

`vercel.json`

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

La app es una **SPA** (*Single Page Application*): en realidad solo existe `index.html`, y es Angular quien cambia el contenido según la URL. Si alguien entra directamente a `.../productos`, Vercel buscaría un fichero llamado `productos` y daría **404**. Esta regla dice: "para **cualquier** URL (`/(.*)`), sirve `index.html`", y Angular ya se encarga del resto.

`"engines": { "node": "24.x" }` en `package.json`: Angular 22 necesita un Node moderno; así Vercel usa Node 24.

`capacitor.config.ts`

```
const config: CapacitorConfig = {
  appId: 'io.ionic.corporateapp',   // identificador único de la app Android
   (como un paquete Java)
  appName: 'corporateApp',          // nombre que se verá en el móvil
  webDir: 'www'                     // carpeta donde está la app compilada
   que se meterá en el móvil
};
```

8.13 Nosotros y Contacto

De momento solo tienen la cabecera con el botón atrás. En los puntos 10–16 del PDF del profe se completan: Nosotros con **geolocalización** (GPS con `@capacitor/geolocation` y la fórmula de Haversine para calcular la distancia a la oficina) y Contacto con un **formulario** (`[(ngModel)]`) que guarda los mensajes con `@capacitor/preferences`.

---

9. Cómo harías la Corporate App tú solo, desde cero

Este es el **orden de trabajo** que tienes que interiorizar. Sirve para casi cualquier app:

```
# 1. Crear el proyecto y entrar
ionic start corporateApp blank --type=angular --standalone
cd corporateApp

# 2. Generar las páginas
ionic generate page pages/home --standalone
ionic generate page pages/productos --standalone
ionic generate page pages/nosotros --standalone
ionic generate page pages/contacto --standalone
```

```
# 3. Generar el servicio
ionic generate service services/products

# 4. Arrancar (y dejarlo corriendo mientras programas)
ionic serve
```

Después, a mano, **en este orden** (de lo que no depende de nada a lo que depende de todo):

Paso	Fichero	Qué escribes
5	`app.routes.ts`	Una ruta por página + la redirección de `''` a `home`. Borra la carpeta `home` que trae la plantilla si la duplicaste.
6	`home.page.html` + `.ts`	El `ion-segment` con 3 botones y `routerLink`. Importar `RouterLink` y los `Ion...` en el `.ts`.
7	`models/product.interface.ts`	La interface con los campos.
8	`assets/data/products.json` + `assets/images/`	Los datos y las fotos.
9	`products.service.ts`	`getProducts()` con `fetch` + `await`.
10	`productos.page.ts`	`signal`, constructor con el servicio, `ngOnInit` que llama al servicio.
11	`productos.page.html`	`ion-grid` con la fila cabecera y la fila con `*ngFor`. Importar `NgFor`, `IonGrid`, `IonRow`, `IonCol`.
12	Probar en el navegador	Si sale en blanco: **F12 → Console** y leer el error.

Errores típicos y qué significan:

Error	Causa	Solución
`'ion-grid' is not a known element`	No importaste `IonGrid` en el `.ts`.	Añadirlo al `import` y a `imports: [...]`.
`Can't bind to 'ngForOf'`	Usas `*ngFor` sin importar `NgFor`.	Importar `NgFor` (o usar `@for`).
La tabla sale vacía sin errores	Asignas una propiedad normal después de `await`.	Usar `signal` y `.set()`.
`404 assets/data/products.json`	La ruta o el nombre del fichero están mal.	Revisar que esté en `src/assets/data/`.
`Cannot find module '../models/...'`	Ruta del `import` mal (número de `../`).	Contar las carpetas que hay que subir.

---

10. Git, GitHub y Vercel

> Esto es el resumen rápido. La explicación completa de ramas, comandos y desde qué carpeta lanzarlos está en la sección 21.

10.1 Conceptos

Palabra	Qué es
Git	Programa en tu PC que guarda el **historial** de una carpeta.
Repositorio (repo)	Una carpeta controlada por Git (tiene una carpeta oculta `.git` con el historial).
Commit	Una "foto" del proyecto en un momento, con un mensaje.
Rama (branch)	Una línea de trabajo paralela. La principal es `main`.
GitHub	Web que guarda una copia de tu repositorio en internet.
remote / origin	La dirección del repo en GitHub. `origin` es el nombre por defecto.
push	Subir tus commits a GitHub.
pull	Bajar de GitHub los commits que no tienes.
`.gitignore`	Lista de cosas que Git debe ignorar (`node_modules`, `www`...).
Vercel	Web que coge tu repo de GitHub, ejecuta el build y publica la app con una URL.

10.2 Tu organización (desde hoy)

Cada asignatura es **un repositorio distinto**, en tu cuenta **JorgeGonzalezDl**:

Carpeta en tu PC	Repositorio
`C:\DAM 2\Desarrollo de Interfaces`	`Desarrollo-Interfaces-DAM-2` (el que tiene tu profe)
`C:\DAM 2\Acceso a Datos`	`Acceso-Datos-DAM-2`
`C:\DAM 2\Programacion de Servicios y Procesos`	`Programacion-Servicios-Procesos-DAM-2`
`C:\DAM 2\Unity`	`Unity-DAM-2`

`C:\DAM 2` **no** es un repositorio: los comandos de git se lanzan siempre **dentro** de la carpeta de la asignatura.

10.3 El flujo de cada día

```
cd "C:\DAM 2\Desarrollo de Interfaces"
git status                      # ¿qué he cambiado? (rojo = sin preparar,
   verde = preparado)
git add .                       # preparar TODOS los cambios (el punto =
   "todo")
git commit -m "Mensaje claro"   # hacer la foto con un mensaje
git push                        # subir a GitHub
```

Otros útiles: `git log --oneline` (ver el historial), `git pull` (bajar cambios), `git branch` (ver ramas), `git checkout nombre` (cambiar de rama), `git merge rama` (traer los cambios de otra rama a la actual).

10.4 Vercel

Configuración de tu proyecto (ya hecho): repo `Desarrollo-Interfaces-DAM-2`, *Root Directory* `02-CorporateApp/corporateApp` (dónde está el `package.json`), *Build Command* `npm run build`, *Output Directory* `www`. Cada `git push` a `main` → Vercel vuelve a compilar y publicar solo.

---

11. Glosario

Término	Significado corto
Angular	Framework de Google para hacer aplicaciones web con componentes.
App híbrida	App hecha con tecnología web que funciona en navegador y móvil.
assets	Carpeta de ficheros estáticos (imágenes, JSON) que se copian tal cual.
async / await	Forma de escribir código asíncrono como si fuera secuencial.
Binding	Conexión entre datos del `.ts` y el `.html`.
Build	Compilar el proyecto para producción (genera `www/`).
Callback	Función que pasas a otro para que la llame más tarde.
Capacitor	Herramienta para convertir la web en app nativa y acceder al hardware.
CLI	Programa que se usa con comandos de texto (`ionic`, `ng`, `git`).
Componente	Trozo de interfaz con su `.ts`, `.html` y `.scss`.

Término	Significado corto
Decorador	Etiqueta con `@` que da información extra a una clase (`@Component`).
Dependencia	Librería que necesita el proyecto, o servicio que necesita un componente.
Deploy / despliegue	Publicar la app en internet.
DI (inyección de dependencias)	Angular entrega a cada componente los servicios que pide.
fetch	Función del navegador para hacer peticiones HTTP. Devuelve una Promise.
Framework	Esqueleto con reglas para construir aplicaciones.
Interface (TS)	Molde que define la forma de un objeto.
Interpolación	`{{ dato }}`: escribir un valor en el HTML.
JSON	Formato de texto para datos: `{ "clave": valor }`.
Lazy loading	Cargar el código de una página solo cuando se visita.
Live reload	Recarga automática del navegador al guardar.
ngOnInit	Método que Angular ejecuta al crear el componente.
node_modules	Carpeta con las librerías descargadas por npm. No se sube a GitHub.
npm	Gestor de paquetes de Node.
Pipe	`{{ valor
Promise	Objeto que representa un resultado que llegará más tarde.
Property binding	`[atributo]="dato"`: rellenar un atributo con una variable.
Ruta	Asociación entre una URL (`/productos`) y una página.
Selector	Nombre de etiqueta de un componente (`app-home`).
Servicio	Clase con lógica/datos que usan los componentes. `@Injectable`.
Signal	Caja de valor que avisa a Angular cuando cambia. `.set()` / `()`.
Singleton	Que solo existe una instancia.
SPA	Aplicación de una sola página HTML cuyo contenido cambia con JavaScript.
Standalone	Componente que declara él mismo lo que usa (sin NgModule).
TypeScript	JavaScript con tipos.

Término	Significado corto
Two-way binding	`[(ngModel)]`: el campo y la variable se actualizan mutuamente.
Zoneless	Angular sin zone.js; por eso usamos signals.

---

12. Ejercicios para practicar (con soluciones)

Hazlos sobre la Corporate App, con `ionic serve` corriendo. Si rompes algo, `git checkout .` deshace todos los cambios no guardados en un commit.

Ejercicio 1 (fácil). Añade un 4º producto al JSON (por ejemplo, un ratón). Usa una foto que ya exista. ¿Qué ficheros has tenido que tocar?

Ejercicio 2 (fácil). En la Home, cambia el título "Corporate App" por el nombre de tu empresa inventada, pero sacándolo de una variable del `.ts` con interpolación.

Ejercicio 3 (medio). Añade el campo `categoria` (texto) a los productos y muéstralo como una columna más del grid. Ajusta los `size` para que sigan sumando 12.

Ejercicio 4 (medio). Debajo de la tabla, muestra el texto "Total de unidades: X", calculado a partir de la lista.

Ejercicio 5 (medio). Muestra el precio en rojo si quedan menos de 10 unidades.

Ejercicio 6 (más difícil). Crea una página nueva `ofertas` con su ruta y un 4º botón en el menú de la Home que lleve a ella.

▶ **Soluciones** (intenta primero sin mirar)

---

13. Chuleta para mañana en clase

Si el profe pregunta, esto es lo que tienes que saber decir **con tus palabras**:

1. **¿Qué es Ionic / Angular / Capacitor?** Ionic = componentes visuales de móvil; Angular = el framework que organiza la app en componentes, rutas y servicios; Capacitor = convierte la web en app nativa y da acceso al hardware.
2. **¿Qué es un componente standalone?** Un componente que declara él mismo en `imports` lo que usa, sin necesidad de un `NgModule`.
3. **¿Qué es el lazy loading?** Cargar cada página solo cuando se visita, con `loadComponent` e `import()` dinámico.
4. **¿Cómo llegan los productos a la pantalla?** JSON en `assets` → `ProductsService` lo lee con `fetch` → `ProductosPage` recibe el servicio por inyección de dependencias y en `ngOnInit` guarda los productos en un signal → el HTML los recorre con `*ngFor` y los muestra con interpolación y `[src]`.
5. **¿Qué es la inyección de dependencias?** Que la página no crea el servicio con `new`, sino que lo pide en el constructor (o con `inject()`) y Angular se lo da. Con `providedIn: 'root'` hay una sola instancia para toda la app.
6. **¿Por qué usas `signal` y no `products: any[]` como en el PDF?** Porque Angular 22 no usa zone.js y no se enteraría de un cambio hecho después de un `await`; el signal le avisa.
7. **Diferencia entre `{{ }}`, `[ ]`, `( )` y `[( )]`.** Interpolación, property binding, event binding y two-way binding (ver 5.3).
8. **dependencies vs devDependencies.** Las primeras van en la app final; las segundas solo se usan para desarrollar/compilar.
9. **¿Qué hace `vercel.json`?** Redirige todas las URLs a `index.html` porque es una SPA.
10. **¿Por qué no se sube `node_modules`?** Porque pesa mucho y se regenera con `npm install` a partir de `package.json` y `package-lock.json`.

Siguiente en la práctica (lo que probablemente toque mañana): punto 10 del PDF, geolocalización en la página Nosotros (`npm install @capacitor/geolocation`, un `GeolocationService` y la fórmula de Haversine).

PARTE 2 — Consumo de APIs REST

> Material de clase del 05-10-2026: dossier **"Consumo de APIs en arquitecturas frontend con Ionic y Angular"** y **"Actividad guiada — Ionic + Angular Standalone: consumo de API REST"**.
>
> **Antes de esta parte** tienes que tener claro el punto 3.10 (Promise, `async/await`) y el punto 4. Aquí aparece una cosa nueva que se parece mucho: el **Observable**.

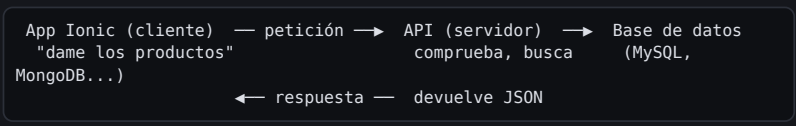
---

14. ¿Qué es una API y qué es REST?

14.1 API

API (*Application Programming Interface*) = un conjunto de reglas para que **dos programas se hablen** sin que uno tenga que saber cómo está hecho el otro por dentro.

Analogía del restaurante (otra vez): tú (la app) no entras en la cocina (la base de datos). Hablas con el **camarero** (la API): le pides algo de la carta (una petición), él va a cocina y te trae el plato (la respuesta). No sabes ni te importa cómo se cocina.



- **Cliente** (*frontend*): lo que ve el usuario. Tu app Ionic.
- **Servidor** (*backend*): el programa que tiene los datos. Puede estar hecho en Node, Spring (Java), Django (Python)...
- Hasta ahora tus datos estaban en un JSON **dentro** de la app (`assets/data/products.json`). Ahora vienen de un **servidor de internet**.

14.2 Por qué se separa frontend y backend (lo que pide el dossier)

Ventaja	Qué significa en cristiano
Desacoplamiento (*separation of concerns*)	El frontend y el backend se programan, se publican y se actualizan por separado.
Reutilización omnicanal	La misma API sirve a la web, a la app móvil y a otros programas.
Escalabilidad	Si hay mucha gente, se refuerza el servidor sin tocar la app.
Seguridad	Las contraseñas de la base de datos y la lógica importante se quedan en el servidor; la app solo ve lo que la API le deja ver (con tokens como **JWT** u **OAuth**).

14.3 Tipos de API

Tipo	Cómo es	Dónde se usa
REST	Usa HTTP normal: URLs + verbos (GET, POST...) + JSON.	Lo estándar hoy. **La que usamos.**
GraphQL	Una sola URL; el cliente pide **exactamente** los campos que quiere. Evita el *over-fetching* (recibir datos de más) y el *under-fetching* (recibir de menos y tener que pedir otra vez).	Creada por Facebook.
SOAP	Basada en XML, muy estricta y pesada.	Bancos, sistemas antiguos (*legacy*).
gRPC	Muy rápida, binaria (HTTP/2 + Protocol Buffers).	Comunicación entre microservicios (Google).

14.4 REST: las 4 reglas

Una API es **REST** si cumple estas restricciones:

1. **Cliente-servidor**: el cliente se ocupa de la interfaz; el servidor, de los datos.
2. **Stateless (sin estado)**: cada petición lleva **toda** la información necesaria. El servidor no "se acuerda" de ti entre una petición y otra (por eso, si hay login, el token se manda en **cada** petición).
3. **Caché**: las respuestas dicen si se pueden guardar temporalmente para no volver a pedirlas.
4. **Interfaz uniforme**: cada cosa (**recurso**) tiene su URL (`/api/usuarios/123`) y se manipula con los verbos HTTP

estándar, devolviendo JSON (o XML).

REST vs RESTful: *REST* es el **conjunto de reglas** (el estilo, definido por Roy Fielding). *RESTful* es el **adjetivo** para una API concreta que las cumple: "he hecho una API RESTful en Node".

14.5 HTTP en lo justo

Anatomía de una URL de API:

```
https://dummyjson.com/products/1?select=title,price
└──┘   └─────────┘ └──────┘ └┘ └──────────────────┘
protocolo  dominio     recurso   id   parámetros (query string)
```

Cada URL a la que se le pueden hacer peticiones se llama **endpoint**.

Verbos HTTP (qué quieres hacer) — se corresponden con el **CRUD**:

Verbo	CRUD	Ejemplo
GET	Read (leer)	`GET /products` → lista; `GET /products/1` → el producto 1
POST	Create (crear)	`POST /products` con un JSON en el cuerpo → crea uno nuevo
PUT / **PATCH**	Update (modificar)	`PUT /products/1` → cambia el producto 1 entero / `PATCH` solo algunos campos
DELETE	Delete (borrar)	`DELETE /products/1`

Códigos de estado (cómo ha ido): `200` OK · `201` creado · `400` petición mal hecha · `401` no autenticado · `403` prohibido · `404` no existe · `500` error del servidor. Regla fácil: **2xx bien, 4xx culpa tuya (cliente), 5xx culpa del servidor**.

> **Pruébalo:** abre https://dummyjson.com/products en el navegador. Lo que ves es la respuesta JSON de un GET. Prueba también `https://dummyjson.com/products/1` y `https://dummyjson.com/products/9999` (un 404).

---

15. Promise vs Observable (`HttpClient` y RxJS)

> Lo que te dijo tu profe es correcto: **en el proyecto de Angular + API no vas a usar `async` ni `await`**, porque los datos no llegan con una **Promise** sino con un **Observable**. Esta sección explica qué es eso, por qué no lleva `await` y cómo se usa. Es la más importante de la Parte 2.
>
> Requisito: tener claro qué es una Promise y `async/await` (sección 3.10). Como dices que eso ya lo tienes, vamos a construir sobre ello.

15.1 Dos formas de pedir datos en Angular

Herramienta	De dónde viene	Qué devuelve	Cómo se recibe el dato
`fetch(url)`	Del **navegador** (JavaScript normal)	Una **Promise**	`await` o `.then()`
`this.http.get(url)`	De **Angular** (`HttpClient`)	Un **Observable**	`.subscribe()` o pipe `async`

Hasta ahora (Corporate App) usábamos `fetch` + `async/await`. **A partir de ahora**, como pide el profe, usaremos `HttpClient`, que es la herramienta oficial de Angular para hablar con APIs. `HttpClient` es un **servicio** de Angular, así que se consigue por **inyección de dependencias** (sección 5.9): `private http = inject(HttpClient);`.

15.2 ¿Qué es un Observable? (con analogías)

Repaso de la Promise: es el **avisador del restaurante**. Pides, te dan el avisador, vibra **una vez** con tu comida, y se acabó.

Un Observable es como una suscripción a un canal de YouTube:

YouTube	Observable
El canal existe, pero **no te llega nada** si no estás suscrito.	El Observable existe, pero **no hace nada** hasta que alguien se suscribe.
Le das a **Suscribirse**.	Llamas a `.subscribe(...)`.
Te llegan notificaciones de vídeos nuevos (pueden ser muchas, a lo largo del tiempo).	Te llegan **valores** (`next`). Pueden ser varios.

YouTube	Observable
Si el canal tiene un problema, te llega un aviso.	Te llega un **error** (`error`).
El canal cierra: ya no llegarán más vídeos.	El Observable **termina** (`complete`).
Te das de baja.	`unsubscribe()`.

Hay **dos diferencias de fondo** con la Promise:

1. Un Observable puede emitir muchos valores; una Promise, solo uno. Por ejemplo, un Observable podría emitir la posición del GPS cada segundo, o cada letra que el usuario escribe en un buscador. **Una petición HTTP solo emite un valor** (la respuesta) y termina, así que en la práctica, para APIs, se usa casi igual que una Promise.

2. Un Observable es "perezoso" (*lazy*): no empieza hasta que te suscribes.

Esta es la diferencia que más importa. Mira:

```
// PROMISE: la petición se envía YA, en esta línea, aunque nadie la espere
const promesa = fetch('https://dummyjson.com/products');

// OBSERVABLE: aquí NO se envía nada. Solo se ha preparado la petición.
const observable = this.http.get('https://dummyjson.com/products');

observable.subscribe(...);   // ← AHORA sí se envía la petición
```

Analogía de la receta: un Observable es como una **receta de cocina** escrita en un papel. Tenerla no te da de comer; alguien tiene que **cocinarla** (suscribirse). Una Promise es un plato que **ya se está cocinando** desde que lo pides.

> Si llamas a `this.http.get(url)` y no te suscribes, **la petición nunca se envía**. En la pestaña *Network* del F12 ni siquiera aparece. Es el error número 1 con Observables.

15.3 ¿Por qué NO se usa `async` / `await`?

Porque `await` **solo sabe esperar Promises**. Un Observable no es una Promise, así que `await` no sabe qué hacer con él:

```
async cargar() {
  const datos = await this.http.get(url);   // ❌ NO espera nada
  console.log(datos);                        // muestra el Observable (la
  "receta"), no los productos
}
```

`await` ve algo que no es una Promise, lo devuelve tal cual y sigue. `datos` es el Observable sin abrir, y además **la petición ni se ha enviado** (nadie se ha suscrito).

Por eso con `HttpClient` el patrón es otro: **no hay `async`, no hay `await`, hay `subscribe`**.

(Si algún día necesitas convertir un Observable en Promise para usar `await`, existe `await firstValueFrom(this.http.get(url))`, de `'rxjs'`. Pero **no** es lo que pide el profe.)

15.4 `subscribe` paso a paso

```
this.productService.getProducts()          // ① devuelve un Observable (la
  receta, aún sin cocinar)
  .subscribe({                             // ② me suscribo → AHORA se
  envía la petición
    next: (response) => {                  // ③ esta función se ejecuta
  cuando LLEGA la respuesta
      this.products.set(response.products);
    },
    error: (err) => {                      // ④ esta, si FALLA (sin
  internet, 404, 500...)
      console.error(err);
    },
    complete: () => {                      // ⑤ (opcional) cuando el
  Observable termina
      console.log('terminado');
    }
  });
console.log('Esto sale ANTES que los productos');   // ⑥
```

Lo que le pasas a `subscribe` es un **objeto** `{ }` con hasta tres funciones flecha:

Clave	Cuándo la llama RxJS	Equivale en Promise a...
`next`	Cada vez que llega un valor (en HTTP: una vez, con la respuesta)	`.then(...)` / el resultado del `await`
`error`	Si algo falla	`.catch(...)` / el `catch` del `try`
`complete`	Cuando ya no van a llegar más valores	`.finally(...)` (más o menos)

Orden en el tiempo (igual que con `await`, el programa no se bloquea):

```
① se prepara el Observable
② subscribe → sale la petición hacia dummyjson.com
⑥ "Esto sale ANTES que los productos"     ← el código sigue sin esperar
   ... (unos milisegundos después) ...
③ next: llega la respuesta → se guardan los productos → la tabla se pinta
⑤ complete
```

Tú **no esperas** a los datos: dejas preparado **qué hacer cuando lleguen** (la función `next`), y RxJS la llama él solo en su momento. (Fíjate que esto es la idea de *callback* de la sección 4: le pasas una función a otro para que la llame más tarde. Por eso merecía la pena entenderlo).

15.5 La misma tarea de tres formas (compáralas)

Objetivo: cargar los productos de dummyjson en la página.

A) Lo que hacíamos: `fetch` + `async/await` (Promise)

```ts
// servicio
async getProducts(): Promise<ProductsResponse> {
  const response = await fetch('https://dummyjson.com/products');
  return await response.json();
}

// página
async ngOnInit() {
  const response = await this.productService.getProducts();
  this.products.set(response.products);
}
```

B) Lo que pide el profe: `HttpClient` + `subscribe` (Observable)

```ts
// servicio: SIN async, SIN await, SIN .json()
getProducts(): Observable<ProductsResponse> {
  return this.http.get<ProductsResponse>('https://dummyjson.com/products');
}

// página: SIN async, SIN await
ngOnInit() {
  this.productService.getProducts().subscribe({
    next: (response) => this.products.set(response.products),
    error: (err) => console.error(err)
  });
}
```

C) La más corta: `HttpClient` + pipe `async` en el HTML (Observable)

```
// página: ni siquiera hay subscribe
products$ = this.productService.getProducts();
```

```
@if (products$ | async; as response) {
  @for (product of response.products; track product.id) { <p>{{
   product.title }}</p> }
}
```

	A) fetch	B) subscribe	C) pipe async
`async` / `await`	Sí	**No**	**No**
Convertir a JSON	A mano (`.json()`)	Automático	Automático
Dónde se recibe el dato	Después del `await`	En `next:`	En el HTML
¿Necesita signal en Angular 22?	Sí	Sí	No (el pipe avisa solo)
Errores	`try/catch`	`error:`	(más avanzado: `catchError`)

B es la que usa la actividad guiada. C es la que usa el dossier (sección 17). A ya no la usaremos para APIs.

15.6 El servicio, línea a línea

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { ProductsResponse } from '../models/product.model';

@Injectable({ providedIn: 'root' })
export class ProductService {
  private http = inject(HttpClient);                          // DI: pido el
   HttpClient de Angular
  private apiUrl = 'https://dummyjson.com/products';

  getProducts(): Observable<ProductsResponse> {               // devuelve "una
   receta que dará un ProductsResponse"
    return this.http.get<ProductsResponse>(this.apiUrl);      // prepara un
   GET; NO lo envía
  }
}
```

- `this.http.get(url)` = "prepara una petición GET a esta URL". Existen también `.post()`, `.put()`, `.delete()` para los otros verbos HTTP (sección 14.5).
- `<ProductsResponse>` después de `get` es un **genérico** (como `List<String>` en Java): le dice a TypeScript qué forma tendrá

la respuesta, para que luego te autocomplete `response.products`. Ojo: es una promesa **tuya**; TypeScript no comprueba lo que manda de verdad el servidor.
- `Observable<ProductsResponse>` como tipo de retorno = "devuelvo un Observable que, cuando alguien se suscriba, emitirá un `ProductsResponse`".
- El servicio **no se suscribe**: devuelve la receta y **quien la use** (la página) decide cuándo cocinarla.

15.7 Activar `HttpClient`: `provideHttpClient()`

Para que `inject(HttpClient)` funcione, Angular tiene que tener registrado el "proveedor" de `HttpClient`. Se hace en la configuración global:

```
providers: [
  // ...
  provideHttpClient(),      // de '@angular/common/http'
]
```

En tu plantilla de Ionic eso va en `main.ts` (ver 16.3). En Angular 22 funciona incluso sin ponerlo, pero ponlo siempre: lo pide la checklist del profe.

15.8 La convención del `$`

Cuando una variable guarda un Observable, por costumbre se le pone **un `$` al final**: `users$`, `products$`. No hace nada especial: solo avisa a quien lee el código de que "esto es un Observable, hay que suscribirse para tener el dato".

15.9 El pipe `async` (forma C)

En vez de suscribirte tú en el `.ts`, dejas que **la plantilla se suscriba sola**:

```
@if (users$ | async; as users) {        <!-- ① se suscribe ② espera ③
   guarda el resultado en "users" -->
  @for (user of users; track user.id) { <p>{{ user.name }}</p> }
} @else {
  <p>Cargando...</p>                     <!-- mientras no ha llegado nada --
  >
}
```

- `users$ | async` = "suscríbete a `users$` y dame el último valor que haya llegado" (mientras no llega nada, vale `null`, y por eso entra en el `@else`).
- `as users` = "a ese valor llámalo `users` dentro del bloque".
- Ventajas: no escribes `subscribe`, Angular se **desuscribe solo** al salir de la página (evita *memory leaks*, fugas de memoria) y avisa él mismo a Angular para repintar (no necesita signals).
- Hay que importar `AsyncPipe` (de `'@angular/common'`) en el componente.

15.10 Errores típicos con Observables

Error	Qué pasa	Solución
No suscribirse	La petición no se envía; la pantalla no muestra nada; en F12 → *Network* no aparece	Añadir `.subscribe({...})` o usar el pipe `async`
Usar `await` con un Observable	`datos` es el Observable, no los productos	Usar `subscribe`
Intentar **devolver** el dato desde dentro de `subscribe`	El `return` dentro de `next` no sale de tu método: se pierde	Guardarlo en una variable/signal dentro de `next`
Variable normal dentro de `next` (Angular 22)	Se queda en "Cargando" para siempre (comprobado)	Usar **signals** y `.set()`
Olvidar `response.products`	Intentas hacer `@for` sobre un objeto, no un array	Coger la lista de dentro del objeto envoltorio

El tercero merece un ejemplo, porque es muy común:

```
// ❌ MAL: este método no devuelve los productos
getLista() {
  this.productService.getProducts().subscribe({
    next: (r) => { return r.products; }     // este return es de la función flecha, no de getLista
  });
}                                           // getLista termina sin devolver nada

// ✅ BIEN: guardarlo donde la plantilla lo pueda leer
loadProducts() {
  this.productService.getProducts().subscribe({
    next: (r) => this.products.set(r.products)
  });
}
```

15.11 Tabla final: Promise vs Observable

	Promise	Observable
De dónde viene	JavaScript estándar	Librería **RxJS** (viene con Angular)
Quién lo usa	`fetch`, funciones `async`	`HttpClient` de Angular
Cuántos valores	**Uno**	**Cero, uno o muchos** a lo largo del tiempo
¿Empieza solo?	**Sí**, en cuanto lo creas	**No**, hasta que te suscribes (*lazy*)
Recibir el valor	`await` o `.then(...)`	`.subscribe({ next })` o pipe `async`
Recibir el error	`try/catch` o `.catch(...)`	`error:` en el subscribe
¿Se puede cancelar?	No	Sí (`unsubscribe()`)
Tipo en TypeScript	`Promise<Product[]>`	`Observable<Product[]>`
Analogía	El avisador del restaurante	La suscripción a un canal de YouTube

15.12 `fetch` vs `HttpClient`

	`fetch`	`HttpClient`
Qué es	Función del navegador	Servicio de Angular (se **inyecta**)
Devuelve	Promise	Observable
Convertir a JSON	A mano: `await response.json()`	Automático
Errores 404/500	**No** cuentan como error (hay que mirar `response.ok`)	**Sí**, van a `error:`
Extras	—	Tipado con `<T>`, interceptores (p. ej. añadir un token a todas las peticiones)

15.13 Para decirlo en clase

> *"Con `HttpClient`, Angular no devuelve una Promise sino un Observable de RxJS. Un Observable no hace la petición hasta que alguien se suscribe, y puede emitir varios valores. Por eso no se usa `await`: se llama a `subscribe` pasándole una función `next` para cuando llegan los datos y otra `error` para cuando falla. Otra*

> *opción es usar el pipe `async` en la plantilla, que se suscribe y se desuscribe solo."*

---

16. Actividad guiada: `dam2-productos` (dummyjson) línea a línea

Objetivo: app Ionic + Angular standalone que pide los productos a `https://dummyjson.com/products`, los muestra en una **tabla**, tiene dos rutas (`/inicio` y `/productos`), se sube a GitHub en una rama `desarrollo` y se publica en **Vercel**.

Es **la misma idea que tu Corporate App**, pero los datos vienen de una API de internet en vez de un JSON local, y se usa `HttpClient` en vez de `fetch`.

```
src/app/
├── models/product.model.ts          → interfaces Product y ProductsResponse
├── services/product.service.ts      → pide los datos con HttpClient
├── pages/inicio/inicio.page.ts      → portada con un botón
├── pages/productos/productos.page.ts→ tabla con los productos
├── app.component.ts                 → raíz con <ion-router-outlet>
├── app.routes.ts                    → rutas
└── app.config.ts                    → configuración global
(provideHttpClient)
```

⚠️ 16.0 Tres avisos antes de copiar el PDF (comprobados con tu versión)

Tu Ionic/Angular es más nuevo que el del PDF. He probado el código del PDF y pasa esto:

En el PDF pone	En tu versión (Ionic 9 + Angular 22)	Qué hacer
`from '@ionic/angular/standalone'`	**No existe** en Ionic 9 → error al compilar.	Importar de `'@ionic/angular'` (como en tu Corporate App).
`products: Product[] = []`, `loading = false` y se cambian dentro de `subscribe`	Angular 22 no usa zone.js → **la página se queda en "Cargando" para siempre** (comprobado).	Usar **signals**: `products = signal<Product[]>([])` y `.set(...)`. Ver versión corregida en 16.7.

En el PDF pone	En tu versión (Ionic 9 + Angular 22)	Qué hacer
`app.config.ts` con `provideHttpClient()`	La plantilla de Ionic **no trae** `app.config.ts`: los *providers* están en `main.ts`.	Añadir `provideHttpClient()` a los `providers` de `main.ts` (o crear `app.config.ts`, ver 16.3).

16.1 Crear el proyecto

```
npm install -g @ionic/cli                          # actualizar la CLI de
  Ionic (global)
ionic start dam2-productos blank --type=angular    # crear proyecto en
  blanco
cd dam2-productos
ionic serve                                        # comprobar en
  http://localhost:8100
```

16.2 Standalone vs módulos (lo que explica el PDF)

```
Angular "tradicional"                  Angular standalone (el nuestro)
AppModule                              Application
 ├── declarations (componentes)         ├── app.config.ts  (providers
globales)
 ├── imports (otros módulos)            ├── app.routes.ts  (rutas)
 └── providers (servicios)              └── cada componente con sus propios
imports
```

Antes, todo se "apuntaba" en un fichero central `AppModule` (`@NgModule`). Ahora **cada componente declara lo que usa** en su `imports: [...]`, y lo global va en `app.config.ts` (o en `main.ts`). **Checklist del profe:** no debe existir `app.module.ts` ni `@NgModule`.

Sobre `imports: [CommonModule, IonicModule]` que aparece en el PDF: funciona, pero **mete todo** Ionic y todo `CommonModule`. Lo moderno (y lo que pide luego el propio PDF) es importar **solo lo que usas**: `IonHeader`, `IonButton`, `CurrencyPipe`...

16.3 Activar `HttpClient`

Lo que pone el PDF (`src/app/app.config.ts`):

```
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {   // objeto de configuración
  de la app
  providers: [
    provideHttpClient()                         // "registra HttpClient para
   poder inyectarlo"
```

```
  ]
};
```

En **tu** plantilla Ionic los providers están en `main.ts`, así que lo más simple es añadirlo ahí:

```
import { provideHttpClient } from '@angular/common/http';   // ← añadir este
  import

bootstrapApplication(AppComponent, {
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
    provideIonicAngular(),
    provideRouter(routes, withPreloading(PreloadAllModules)),
    provideHttpClient(),                                    // ← y esta
   línea
  ],
});
```

> Nota: en Angular 22 `HttpClient` ya funciona aunque no pongas `provideHttpClient()` (lo he probado), pero **ponlo igualmente**: lo pide la checklist y es donde se configuran cosas como los interceptores.

Es **inyección de dependencias** otra vez (sección 5.9): `provideHttpClient()` registra el "proveedor" de `HttpClient` en el inyector raíz, y luego el servicio lo pide con `inject(HttpClient)`.

16.4 El modelo: `models/product.model.ts`

```
export interface Product {
  id: number;
  title: string;                // nombre del producto
  description: string;
  category: string;
  price: number;
  discountPercentage: number;   // % de descuento
  rating: number;               // valoración (0-5)
  stock: number;                // unidades disponibles
  brand: string;                // marca (algunos productos NO la traen)
  thumbnail: string;            // URL de la imagen pequeña
}

export interface ProductsResponse {
  products: Product[];          // la lista de productos
  total: number;                // cuántos productos hay en total en la API
   (194)
  skip: number;                 // cuántos se ha saltado (para paginar)
  limit: number;                // cuántos devuelve por petición (30 por
   defecto)
}
```

¿Por qué dos interfaces? Porque la API **no devuelve directamente un array**, sino un objeto que **envuelve** el array (abre https://dummyjson.com/products y lo verás):

```
{
  "products": [ { "id": 1, "title": "Essence Mascara Lash Princess", ... },
  ... ],
  "total": 194,
  "skip": 0,
  "limit": 30
}
```

Por eso luego hay que coger `response.products`. Los nombres de los campos de la interface tienen que ser **exactamente** los del JSON (en inglés, como los manda la API). La API manda más campos de los que pone la interface (`tags`, `reviews`, `images`...) y no pasa nada: la interface solo describe los que vamos a usar.

16.5 El servicio: `services/product.service.ts`

```
ionic generate service services/product
```

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';       // el cliente HTTP
  de Angular
import { Observable } from 'rxjs';                        // el tipo
  Observable
import { ProductsResponse } from '../models/product.model';

@Injectable({
  providedIn: 'root'                                      // un servicio
  para toda la app
})
export class ProductService {

  private http = inject(HttpClient);                      // DI con
  inject(): "dame un HttpClient"

  private apiUrl = 'https://dummyjson.com/products';      // el endpoint

  getProducts(): Observable<ProductsResponse> {           // devuelve un
  Observable (aún no pide nada)
    return this.http.get<ProductsResponse>(this.apiUrl);  // prepara un GET
  a la URL
  }
}
```

- `inject(HttpClient)` = lo mismo que `constructor(private http: HttpClient) {}`. El PDF lo usa para enseñarte `inject()` (lo viste en la sección 5.9).

- Fíjate: **no hay** `async` **ni** `await`. El servicio devuelve el Observable "sin abrir", y quien lo use se suscribirá.
- **Regla del dossier:** las peticiones HTTP van **siempre en un servicio**, nunca directamente en la página.

16.6 Rutas: `app.routes.ts`

```
export const routes: Routes = [
  { path: '', redirectTo: 'inicio', pathMatch: 'full' },            // / →
   /inicio
  { path: 'inicio', loadComponent: () =>
   import('./pages/inicio/inicio.page').then(m => m.InicioPage) },
  { path: 'productos', loadComponent: () =>
   import('./pages/productos/productos.page').then(m => m.ProductosPage) },
  { path: '**', redirectTo: 'inicio' }                              //
   cualquier otra URL → /inicio
];
```

Lo nuevo es `'**'` (comodín, *wildcard*): significa "cualquier ruta que no coincida con las anteriores". Sirve para que una URL inventada (`/patata`) no deje la pantalla en blanco. **Tiene que ir la última**, porque Angular mira las rutas de arriba abajo y se queda con la primera que encaje.

16.7 La página Productos (versión corregida para tu Angular)

`productos.page.ts`

```
import { Component, OnInit, inject, signal } from '@angular/core';
import { CurrencyPipe } from '@angular/common';                     // para
   | currency
import {
  IonHeader, IonToolbar, IonTitle, IonContent, IonButtons, IonBackButton,
  IonSpinner, IonCard, IonCardHeader, IonCardTitle, IonCardContent,
   IonButton,
} from '@ionic/angular';                                            // ← sin
   /standalone
import { Product, ProductsResponse } from '../../models/product.model';
import { ProductService } from '../../services/product.service';

@Component({
  selector: 'app-productos',
  templateUrl: './productos.page.html',
  styleUrls: ['./productos.page.scss'],
  standalone: true,
  imports: [
    CurrencyPipe,
    IonHeader, IonToolbar, IonTitle, IonContent, IonButtons, IonBackButton,
    IonSpinner, IonCard, IonCardHeader, IonCardTitle, IonCardContent,
   IonButton,
  ],
})
export class ProductosPage implements OnInit {
```

```
  private productService = inject(ProductService);   // DI

  products = signal<Product[]>([]);   // la lista          (PDF: products:
   Product[] = [])
  total = signal(0);                  // total en la API   (PDF: total = 0)
  loading = signal(false);            // ¿cargando?        (PDF: loading =
   false)
  error = signal('');                 // mensaje de error  (PDF: error = '')

  ngOnInit(): void {                  // void = no devuelve nada
    this.loadProducts();
  }

  loadProducts(): void {
    this.loading.set(true);           // enseña el spinner
    this.error.set('');               // borra errores anteriores (por si es
   un "Reintentar")

    this.productService.getProducts() // Observable (aún sin pedir)
      .subscribe({                    // ← aquí se hace la petición de
   verdad
        next: (response: ProductsResponse) => {    // llegó bien
          this.products.set(response.products);    // el array está DENTRO
   de response
          this.total.set(response.total);
          this.loading.set(false);
        },
        error: (error) => {                        // falló (sin internet,
   404, 500...)
          console.error(error);
          this.error.set('No se han podido cargar los productos.');
          this.loading.set(false);
        }
      });
  }
}
```

La página tiene **tres estados**: *cargando*, *error* y *correcto*. Es lo que pide la checklist ("Estado de carga / correcto / error / botón de reintento").

`productos.page.html` (con `@if` / `@for`, la sintaxis que recomienda el profe)

```
<ion-header>
  <ion-toolbar>
    <ion-buttons slot="start">
      <ion-back-button defaultHref="/inicio"></ion-back-button>
    </ion-buttons>
    <ion-title>Productos</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">          <!-- ion-padding: margen
   interior de Ionic -->
  <h1>Listado de productos</h1>
```

```
@if (loading()) {                          <!-- ESTADO 1: cargando -->
  <div class="loading">
    <ion-spinner></ion-spinner>            <!-- ruedecita girando -->
    <p>Cargando productos...</p>
  </div>
}

@if (error()) {                            <!-- ESTADO 2: error (un texto
no vacío cuenta como true) -->
  <ion-card>
    <ion-card-header>
      <ion-card-title>Error</ion-card-title>
    </ion-card-header>
    <ion-card-content>
      <p>{{ error() }}</p>
      <ion-button (click)="loadProducts()">Reintentar</ion-button>   <!--
event binding: vuelve a pedir -->
    </ion-card-content>
  </ion-card>
}

@if (!loading() && !error()) {             <!-- ESTADO 3: correcto. && =
Y, ! = NO -->
  <p>Productos cargados: <strong>{{ products().length }}</strong> de {{
total() }}</p>

  <div class="table-container">
    <table>                                <!-- tabla HTML normal -->
      <thead>                              <!-- cabecera de la tabla -->
        <tr>                               <!-- tr = fila (table row) -->
          <th>ID</th>                      <!-- th = celda de cabecera -->
          <th>Producto</th>
          <th>Categoría</th>
          <th>Marca</th>
          <th>Precio</th>
          <th>Valoración</th>
          <th>Stock</th>
        </tr>
      </thead>
      <tbody>                              <!-- cuerpo de la tabla -->
        @for (product of products(); track product.id) {
          <tr>
            <td>{{ product.id }}</td>      <!-- td = celda normal (table
data) -->
            <td>
              <div class="product">
                <img [src]="product.thumbnail" [alt]="product.title">
                <span>{{ product.title }}</span>
              </div>
            </td>
            <td>{{ product.category }}</td>
            <td>{{ product.brand || 'Sin marca' }}</td>      <!-- si no
hay marca, pone 'Sin marca' -->
            <td>{{ product.price | currency:'EUR' }}</td>    <!-- pipe de
moneda -->
            <td>⭐ {{ product.rating }}</td>
            <td>{{ product.stock }}</td>
          </tr>
        }
      </tbody>
    </table>
```

```
    </div>
  }
</ion-content>
```

Cosas nuevas:

- `a || b` = "si `a` está vacío / no existe, usa `b`". Algunos productos de dummyjson no traen `brand`, y así no sale la celda vacía.
- `<table>`, `<thead>`, `<tbody>`, `<tr>`, `<th>`, `<td>`: tabla HTML de toda la vida (en la Corporate App usamos el `ion-grid` de Ionic, que es otra forma de hacer tablas).
- `| currency:'EUR'` necesita `CurrencyPipe` en los `imports`. Sale como `€9.99` (formato inglés por defecto).
- Si usas `@if` / `@for`, **no hace falta** `CommonModule`. Si usas `*ngIf` / `*ngFor`, sí (o `NgIf` / `NgFor`).

`productos.page.scss`

```
.loading { text-align: center; padding: 40px; }        /* spinner centrado
  con aire */

.table-container {
  width: 100%;
  overflow-x: auto;              /* si la tabla no cabe (móvil), se hace
  scroll horizontal */
}

table {
  width: 100%;
  min-width: 900px;              /* nunca más estrecha de 900px → en móvil,
  scroll */
  border-collapse: collapse;     /* bordes de celdas pegados (sin doble
  línea) */
}

th, td {                         /* a la vez para th y td */
  padding: 12px;
  border-bottom: 1px solid #ddd;
  text-align: left;
}

th {
  background: var(--ion-color-primary);   /* color principal del tema de
  Ionic (azul) */
  color: white;
}

.product { display: flex; align-items: center; gap: 10px; }   /* imagen y
  nombre en fila, separados 10px */
.product img { width: 50px; height: 50px; object-fit: contain; }  /* imagen
  50x50 sin deformarse */
```

- `var(--ion-color-primary)`: una **variable CSS** de Ionic. Si cambias el color primario del tema, cambia en toda la app.
- `display: flex`: pone los hijos uno al lado del otro.

16.8 Página Inicio y componente raíz

`inicio.page.ts` — solo declara lo que usa su HTML:

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';
import { IonHeader, IonToolbar, IonTitle, IonContent, IonButton } from
  '@ionic/angular';

@Component({
  selector: 'app-inicio',
  templateUrl: './inicio.page.html',
  styleUrls: ['./inicio.page.scss'],
  standalone: true,
  imports: [IonHeader, IonToolbar, IonTitle, IonContent, IonButton,
   RouterLink],
})
export class InicioPage {}
```

`inicio.page.html`

```
<ion-header>
  <ion-toolbar><ion-title>Inicio</ion-title></ion-toolbar>
</ion-header>
<ion-content class="ion-padding">
  <h1>DAM2 - API REST</h1>
  <p>Aplicación Ionic + Angular Standalone para consumir una API REST.</p>
  <ion-button routerLink="/productos">Ver productos</ion-button>   <!--
   necesita RouterLink -->
</ion-content>
```

`app.component.ts` + `.html`: igual que en la Corporate App (sección 8.4): `imports: [IonApp, IonRouterOutlet]` y `<ion-app><ion-router-outlet></ion-router-outlet></ion-app>`.

16.9 GitHub con rama `desarrollo`

```
git init                                          # convierte la carpeta en
  repositorio
git add .
git commit -m "Creación inicial de aplicación Ionic Standalone"
git checkout -b desarrollo                        # crea la rama
  "desarrollo" y se cambia a ella
git branch --show-current                         # comprobar: debe decir
  "desarrollo"
git remote add origin https://github.com/JorgeGonzalezDI/dam2-productos.git
  # conectar con GitHub
git push -u origin desarrollo                     # subir la rama
  desarrollo
```

- `git checkout -b nombre` = crear rama **y** cambiarse a ella (`-b` de *branch*).
- `-u` (*upstream*) en el primer push = "recuerda que esta rama va con `origin/desarrollo`"; los siguientes push son solo `git push`.
- El profe entrará en GitHub → selector de ramas → **desarrollo**.
- El PDF pide un repositorio propio `dam2-productos`. Cuando la hagas, decidimos si va en ese repo nuevo (como pide el PDF) o como carpeta `03-...` de tu repo de la asignatura.

16.10 Vercel

Add New Project → importar `dam2-productos` → **Production Branch: `desarrollo`** (por defecto Vercel usa `main`, hay que cambiarlo) → Deploy. Build `npm run build`, salida `www`, y el `vercel.json` con el *rewrite* a `index.html` (sección 8.12).

Comprobar dos cosas: que `https://TU-APP.vercel.app/` funciona **y** que `https://TU-APP.vercel.app/productos` funciona **recargando la página** (si da 404 → falta el `vercel.json`). Y que los productos se cargan en producción (la API es pública, así que sí).

16.11 Checklist del profe (resumida)

- **Standalone:** sin `app.module.ts`, imports en cada componente, `provideHttpClient()`.
- **API:** se consume `dummyjson.com/products`; existen `Product`, `ProductsResponse`, `ProductService`.
- **Vista:** tabla con ID, nombre, categoría, precio, stock, valoración e imagen.
- **Navegación:** `/inicio`, `/productos`, `/` → `/inicio`, ruta desconocida → `/inicio`, ir y volver.
- **Estados:** carga, correcto, error, botón reintentar.
- **Git/GitHub:** repo, rama `desarrollo`, commits descriptivos.
- **Vercel:** desplegado desde `desarrollo`, URL funcionando, `/productos` funciona directamente.

---

17. El ejemplo del dossier: directorio de usuarios con `async` pipe

El dossier hace lo mismo con usuarios de `https://jsonplaceholder.typicode.com/users` (la API de tu práctica `paradigmas-js`), pero usando el **pipe** `async` en lugar de `subscribe`.

`api.service.ts` — igual que el `ProductService`, con `getUsers(): Observable<User[]>`. Aquí la API sí devuelve **directamente un array**, por eso es `User[]` y no un objeto envoltorio.

`home.page.ts`

```
export class HomePage implements OnInit {
  public users$!: Observable<User[]>;     // $ = es un Observable; ! = "ya
   le daré valor luego, no te quejes"
  private apiService = inject(ApiService);

  ngOnInit() { this.fetchData(); }

  fetchData() {
    this.users$ = this.apiService.getUsers();   // NO se suscribe: guarda el
   Observable "sin abrir"
  }
}
```

- `!` **después del nombre** (*definite assignment*): TypeScript se queja si declaras una propiedad sin valor inicial; el `!` le dice "tranquilo, se la doy en `ngOnInit`".

`home.page.html`

```
<ion-list *ngIf="users$ | async as users; else loading">   <!-- se suscribe;
   mientras no llega → plantilla "loading" -->
  <ion-item *ngFor="let user of users">
    <ion-avatar slot="start">                                  <!-- imagen
   redonda a la izquierda -->
      <img [src]="'https://ui-avatars.com/api/?name=' + user.name"
   alt="Avatar"/>  <!-- texto + variable -->
    </ion-avatar>
    <ion-label>
      <h2>{{ user.name }}</h2>
      <p>{{ user.email }}</p>
      <p><ion-icon name="globe-outline"></ion-icon> {{ user.website }}</p>
    </ion-label>
  </ion-item>
</ion-list>

<ng-template #loading>                                         <!-- plantilla
   con nombre "loading" -->
  <ion-list>
```

```
    <ion-item *ngFor="let i of [1,2,3,4,5]">                <!-- 5 filas
  falsas -->
      <ion-avatar slot="start"><ion-skeleton-text animated></ion-skeleton-
  text></ion-avatar>
      <ion-label>
        <h2><ion-skeleton-text animated style="width: 50%"></ion-skeleton-
  text></h2>
        <p><ion-skeleton-text animated style="width: 80%"></ion-skeleton-
  text></p>
      </ion-label>
    </ion-item>
  </ion-list>
</ng-template>
```

- `users$ | async as users`: el pipe `async` se suscribe y el resultado se llama `users` dentro.
- `; else loading` + `<ng-template #loading>`: "si todavía no hay datos, pinta la plantilla llamada `loading`". `#loading` es una **referencia de plantilla** (un nombre para ese trozo). Con la sintaxis nueva sería `@if (users$ | async; as users) { ... } @else { ... }`.
- `<ion-skeleton-text animated>`: rectángulos grises animados que imitan el contenido mientras carga (*skeleton loading*), como en YouTube o Instagram.
- `'texto' + user.name` dentro de `[src]`: concatena texto con la variable para formar la URL.
- Este ejemplo **sí funciona** en tu Angular 22 sin signals, porque el pipe `async` avisa él mismo a Angular de que hay datos nuevos (lo he comprobado).
- `IonicModule` en los imports del dossier = importar todo Ionic de golpe (forma antigua).

---

18. Ejercicio propuesto: Maestro-Detalle con la API de Rick and Morty

Lo propone el dossier. **Maestro-detalle** (*master-detail*) = una pantalla con la **lista** (maestro) y, al pulsar un elemento, otra pantalla con **todos sus datos** (detalle). Como la app de contactos del móvil.

Requisito	Qué significa / qué vas a usar
1. Interfaces estrictas	Abre https://rickandmortyapi.com/api/character. La respuesta es `{ info: {...}, results: [...] }` → interfaces `Character` y `CharacterResponse` (como `Product` y `ProductsResponse`).

Requisito	Qué significa / qué vas a usar
2. `CharacterService`	Dos métodos: `getCharacters()` → `GET /character` y `getCharacter(id)` → `GET /character/{id}` (un solo personaje).
3. Vista maestra	`<ion-list>` con `<ion-item>` por personaje: `<ion-avatar>` con `image`, `name` y `species`.
4. Navegación con id	Ruta con **parámetro**: `{ path: 'personaje/:id', loadComponent: ... }`. El `:id` es una variable de la URL. En la lista: `[routerLink]="['/personaje', character.id]"`.
5. Vista detalle	La página lee el `id` de la URL, llama a `getCharacter(id)` y muestra `origin.name`, `status`, `gender` en un `<ion-card>`.
6. Errores (avanzado)	Capturar el error (red caída, 404) con `catchError` de RxJS o en el `error:` del subscribe y mostrar un `<ion-toast>` (mensajito que aparece abajo y desaparece).

Pistas para leer el `id` de la URL: tu plantilla Ionic ya trae `withComponentInputBinding()` en `main.ts`, que permite recibir el parámetro de la ruta directamente como una entrada del componente:

```
import { input } from '@angular/core';
export class PersonajePage {
  id = input<string>();          // Angular rellena esto con el :id de la URL ("1", "2"...)
}
```

(La forma clásica es inyectar `ActivatedRoute` y leer `route.snapshot.paramMap.get('id')`.)

Cuando lo vayas a hacer, lo montamos paso a paso.

---

19. Preguntas de repaso de la Parte 2

1. ¿Qué es una API? ¿Y qué la hace REST?
2. ¿Qué verbo HTTP usarías para crear un producto? ¿Y para borrarlo?
3. ¿Qué significa un 404? ¿Y un 500?
4. ¿Qué diferencia hay entre una Promise y un Observable? (di al menos dos)
5. Si llamo a `this.http.get(url)` y no hago nada más, ¿se hace la petición?

6. ¿Por qué el modelo de dummyjson tiene `ProductsResponse` además de `Product`?
7. ¿Para qué sirve la ruta `'**'` y por qué va la última?
8. ¿Qué hace `{{ product.brand || 'Sin marca' }}`?
9. ¿Por qué con Angular 22 hay que usar signals dentro del `subscribe`? ¿Por qué con el pipe `async` no hace falta?
10. ¿Qué tienes que configurar en Vercel para que publique la rama `desarrollo`?

▶ **Respuestas**

PARTE 3 — Tu proyecto `dam2-productos` y Git a fondo

---

20. Tu proyecto `dam2-productos` (actividad 03) línea a línea

📁 `C:\DAM 2\Desarrollo de Interfaces\03-API-REST-Productos\dam2-productos`

La sección 16 explica el PDF del profe. Esta explica **tu** proyecto tal y como está, con las peticiones del cliente que ya están hechas y pistas para las que faltan. Léela con el código abierto al lado.

20.1 Mapa del proyecto

```
dam2-productos/
├── package.json              ← con "engines": { "node": "24.x" } para Vercel
├── vercel.json               ← rewrite a index.html (SPA)
└── src/
    ├── main.ts               ← arranca la app con appConfig
    └── app/
        ├── app.config.ts     ← configuración global: router, Ionic,
HttpClient   (petición 7)
        ├── app.routes.ts     ← /inicio, /productos y '**'
        ├── app.component.ts ← raíz con <ion-router-outlet>
        ├── models/product.model.ts      ← Product, Dimensions,
ProductsResponse
        ├── services/product.service.ts  ← HttpClient → Observable
        └── pages/
            ├── inicio/       ← portada con botón "Ver productos"
            └── productos/    ← tabla + dimensiones + stock valorado + volver
(peticiones 1, 2, 3)
```

El viaje de los datos (compáralo con el de la Corporate App, sección 8.1):

```
dummyjson.com ──HttpClient (Observable)──► ProductService ──inject()──►
ProductosPage ──signal──► tabla HTML
   (internet)                              (prepara el GET)
(subscribe)              (@for)
```

La diferencia con la Corporate App: allí los datos estaban **dentro** de la app (un JSON en `assets`) y se leían con `fetch` + `await`. Aquí vienen **de internet** y se leen con `HttpClient` + `subscribe`.

20.2 `main.ts` + `app.config.ts` (petición 7)

```ts
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { appConfig } from './app/app.config';

bootstrapApplication(AppComponent, appConfig).catch((err) =>
  console.error(err));
```

`main.ts` ahora solo **arranca**: "empieza por `AppComponent` usando la configuración `appConfig`". `.catch(...)` muestra el error en consola si el arranque falla.

```ts
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },   // navegación estilo Ionic
    provideIonicAngular(),                                           // activa Ionic
    provideRouter(routes, withPreloading(PreloadAllModules), withComponentInputBinding()),  // rutas
    provideHttpClient(),                                             // permite inyectar HttpClient
  ],
};
```

- `ApplicationConfig` es el **tipo** de este objeto (TypeScript te avisa si escribes algo que no va).
- `providers` = la lista de cosas que el **inyector** de Angular sabrá crear y repartir (sección 5.9).
- **Respuesta a la petición 7:** existe porque en standalone **no hay** `AppModule`; la configuración global que antes iba en el módulo ahora va en este objeto. La plantilla de Ionic la traía metida en

`main.ts`; la saqué a su propio fichero para separar "arrancar" de "configurar".

20.3 `models/product.model.ts`

```
export interface Dimensions {   // un "sub-molde" para el objeto dimensions
   de la API
  width: number;                // ancho
  height: number;               // alto
  depth: number;                // fondo
}

export interface Product {
  ...
  brand?: string;               // el ? = campo OPCIONAL (algunos productos
   no lo traen)
  dimensions: Dimensions;       // un campo cuyo tipo es otra interface
}
```

- **Interfaces dentro de interfaces:** la API manda `"dimensions": { "width": 15.14, "height": 13.08, "depth": 22.99 }`, que es un objeto dentro del producto. Para describirlo se crea otra interface (`Dimensions`) y se usa como tipo. Luego se accede con puntos: `product.dimensions.width`.
- **`?` = opcional:** `brand?: string` significa "puede estar o no". Por eso en el HTML se usa `product.brand || 'Sin marca'`.

20.4 `services/product.service.ts`

Igual que el de la sección 16.5: `inject(HttpClient)` y un método que devuelve `this.http.get<ProductsResponse>(this.apiUrl)` **sin suscribirse** (devuelve la "receta", sección 15).

20.5 `pages/productos/productos.page.ts`

```
export class ProductosPage implements OnInit {
  private productService = inject(ProductService);   // DI

  products = signal<Product[]>([]);                  // 4 signals = el
   "estado" de la página
  total = signal(0);
  loading = signal(false);
  error = signal('');

  ngOnInit(): void { this.loadProducts(); }          // al abrir la página,
   cargar

  loadProducts(): void {
    this.loading.set(true);                          // → el HTML enseña el
   spinner
    this.error.set('');                              // → borra errores
   anteriores
```

```
    this.productService.getProducts().subscribe({     // → AHORA sale la
  petición
      next: (response: ProductsResponse) => {         // → llegó bien
        this.products.set(response.products);         //   la lista va
  DENTRO de response
        this.total.set(response.total);
        this.loading.set(false);                      //   → quita el
  spinner, pinta la tabla
      },
      error: (err) => {                               // → falló
        console.error(err);
        this.error.set('No se han podido cargar los productos.');   // → el
  HTML enseña la tarjeta de error
        this.loading.set(false);
      },
    });
  }

  stockValorado(p: Product): number {                 // petición 2
    const bruto = p.stock * p.price;
    return bruto - bruto * (p.discountPercentage / 100);
  }
}
```

Cómo leer `stockValorado` paso a paso, con un producto de 99 unidades a 9,99 € y 10 % de descuento:

Línea	Cálculo	Resultado
`const bruto = p.stock * p.price`	99 × 9,99	989,01
`bruto * (p.discountPercentage / 100)`	989,01 × 0,10	98,90 (lo que se descuenta)
`return bruto - ...`	989,01 − 98,90	**890,11**

`discountPercentage` viene como **porcentaje** (10 = 10 %), por eso se divide entre 100.

El estado de la página es una "máquina de 3 estados", controlada por los signals:

`loading()`	`error()`	Qué se ve
`true`	`''`	Spinner "Cargando productos..."
`false`	`'No se han podido…'`	Tarjeta de error con botón Reintentar
`false`	`''`	La tabla

(Un texto vacío `''` cuenta como **falso** en un `@if`; un texto con algo cuenta como **verdadero**.)

20.6 `pages/productos/productos.page.html` (peticiones 1, 2 y 3)

```
<ion-button routerLink="/inicio" fill="outline">Volver a inicio</ion-button>
  <!-- petición 3 -->
```

- `routerLink="/inicio"` navega al pulsar (hace falta `RouterLink` en los `imports` del `.ts`).
- `fill="outline"`: estilo de Ionic, botón solo con borde. Otros: `solid` (relleno), `clear` (sin fondo).

```
<td>{{ product.dimensions.width | number: '1.0-2' }} × {{
  product.dimensions.height | number: '1.0-2' }}</td>  <!-- petición 1 -->
```

- Dos **interpolaciones** seguidas con un `×` en medio.
- `| number: '1.0-2'` es el pipe de números (necesita `DecimalPipe` en `imports`). El formato `'1.0-2'` = mínimo 1 cifra entera, entre 0 y 2 decimales: `15.1400001` → `15.14`.

```
<td>{{ stockValorado(product) | currency: 'EUR' }}</td>   <!-- petición 2 --
  >
```

- Dentro de `{{ }}` se puede **llamar a un método** de la clase pasándole el producto de esa fila. Angular llama a `stockValorado(product)` por cada fila y escribe el resultado formateado como dinero.

20.7 Lo que falta, con pistas (para cuando lo hagas tú)

Petición	Pista
4. About	`ionic generate page pages/about` → añadir la ruta `about` en `app.routes.ts` (¡antes del `'**'`!) → en `inicio.page.html` otro `<ion-button routerLink="/about">` → en `about.page.html` un `<ion-card>` con la reseña y `<a href="https://github.com/JorgeGonzalezDI" target="_blank">`.
6. Paginación	dummyjson acepta `?limit=10&skip=20` (10 productos saltándose los 20 primeros). Un signal `pagina`, botones Anterior/Siguiente con `(click)` que cambian la página y vuelven a llamar a `loadProducts()`. El servicio recibe `skip` y `limit` como parámetros. Total de páginas = `Math.ceil(total / limit)`.
9. Modo oscuro	En `global.scss` cambiar `dark.system.css` por `dark.class.css`. Un `<ion-toggle>` o botón que haga `document.documentElement.classList.toggle('ion-palette-dark')`.

Petición	Pista
10. Reto visual	Cambiar la `<table>` por un `@for` que pinte `<ion-card>` (imagen arriba, título, precio, rating...) dentro de un `<ion-grid>` con `<ion-col size="12" size-md="6" size-lg="3">` (1, 2 o 4 por fila según la pantalla).

---

21. Git y GitHub a fondo: ramas, comandos y desde dónde lanzarlos

21.1 El modelo mental: 4 sitios donde viven tus cambios



1. **Carpeta de trabajo**: tus ficheros normales. Aquí editas.
2. **Staging** (zona de preparación): lo que has marcado con `git add` para la próxima foto.
3. **Repositorio local**: el historial de fotos (commits), guardado en la carpeta oculta `.git`. **Todavía solo en tu PC.**
4. **GitHub** (*remote*, llamado `origin`): la copia en internet. Solo se actualiza con `git push`.

> Error típico: hacer `commit` y pensar que ya está en GitHub. **No:** el commit es local. Hasta que no haces `push`, ni el profe ni Vercel lo ven.

21.2 Desde dónde se lanza cada comando (lo más importante)

Git funciona **dentro de un repositorio**: una carpeta que tiene la carpeta oculta `.git`. Tus repos:

Carpeta (repositorio)	GitHub
`C:\DAM 2\Desarrollo de Interfaces`	`Desarrollo-Interfaces-DAM-2`
`C:\DAM 2\Acceso a Datos`	`Acceso-Datos-DAM-2`
`C:\DAM 2\Programacion de Servicios y Procesos`	`Programacion-Servicios-Procesos-DAM-2`
`C:\DAM 2\Unity`	`Unity-DAM-2`

`C:\DAM 2` **no es** un repositorio. Si lanzas `git status` ahí, sale `fatal: not a git repository`.

Regla: los comandos **de git** van desde la **carpeta de la asignatura**; los comandos **de npm/ionic** van desde la **carpeta del proyecto** (la que tiene `package.json`).

Quiero…	Carpeta desde la que lo lanzo	Comando
Ver cambios, hacer commit, push, cambiar de rama	`C:\DAM 2\Desarrollo de Interfaces`	`git status`, `git add .`, `git commit`, `git push`…
Instalar dependencias	`...\03-API-REST-Productos\dam2-productos`	`npm install`
Arrancar la app	`...\03-API-REST-Productos\dam2-productos`	`ionic serve`
Generar una página	`...\03-API-REST-Productos\dam2-productos`	`ionic generate page pages/about`

(Git en realidad funciona desde cualquier subcarpeta del repo, porque busca el `.git` hacia arriba. Pero acostúmbrate a lanzarlo desde la raíz de la asignatura: así `git add .` coge **todo**, no solo la subcarpeta en la que estés.)

Cómo trabajar cómodo en VS Code: abre **la carpeta del proyecto** (`dam2-productos`) para programar, y abre **una segunda terminal** (icono **+**) para git:

```
# Terminal 1 (proyecto): se queda con ionic serve corriendo
cd "C:\DAM 2\Desarrollo de Interfaces\03-API-REST-Productos\dam2-productos"
ionic serve

# Terminal 2 (git)
cd "C:\DAM 2\Desarrollo de Interfaces"
git status
```

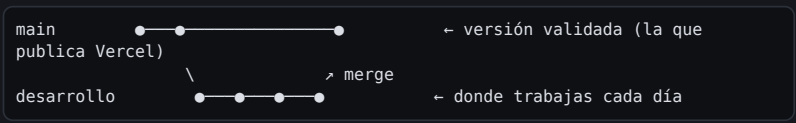
Las comillas en `cd "..."` son **obligatorias** porque la ruta tiene espacios (`DAM 2`).

¿Cómo sé dónde estoy? PowerShell lo pone delante del cursor: `PS C:\DAM 2\Desarrollo de Interfaces>`. Y `git status` te dice la rama en la que estás.

21.3 Ramas: qué son

Una **rama** (*branch*) es una **línea de trabajo paralela** dentro del mismo repositorio.

Analogía de videojuego: es como tener **varias partidas guardadas** del mismo juego. En la partida `main` tienes la versión buena, la que enseñas. En la partida `desarrollo` pruebas cosas; si la lías, la partida `main` sigue intacta. Cuando lo que has hecho en `desarrollo` está bien, lo "pasas" a `main` (*merge*).



- Cada `●` es un commit.
- **HEAD** = "dónde estás ahora mismo" (en qué rama).
- **Al cambiar de rama, los ficheros de tu carpeta CAMBIAN** para mostrar el estado de esa rama. Si en `desarrollo` has creado `about.page.ts` y te cambias a `main` (donde todavía no existe), **el fichero desaparece de la carpeta**. No se ha perdido: vuelve al cambiar a `desarrollo`.

21.4 Comandos de ramas

Comando	Qué hace
`git branch`	Lista las ramas locales; la actual lleva `*`
`git branch -a`	Lista también las de GitHub (`remotes/origin/...`)
`git branch --show-current`	Dice solo el nombre de la rama actual
`git checkout desarrollo`	Cambiarse a la rama `desarrollo` (también vale `git switch desarrollo`)
`git checkout -b nueva`	**Crear** la rama `nueva` y cambiarse a ella (`-b` = *branch*)
`git merge desarrollo`	Traer a la rama **actual** los commits de `desarrollo`

Comando	Qué hace
`git push origin desarrollo`	Subir la rama `desarrollo` a GitHub
`git push -u origin desarrollo`	Lo mismo, y además "recuerda" la relación (*upstream*): luego basta con `git push`
`git branch -d nombre`	Borrar una rama local (ya fusionada)
`git log --oneline --graph --all`	Ver el historial de todas las ramas en forma de árbol

Antes de cambiar de rama, haz commit de lo que tengas a medias. Si no, git puede negarse (*"Your local changes would be overwritten by checkout"*) o llevarse tus cambios a la otra rama.

21.5 Tu flujo de trabajo diario (el que pide el profe)

```
cd "C:\DAM 2\Desarrollo de Interfaces"

# ── 1. Asegurarte de estar en desarrollo ──
git checkout desarrollo
git branch --show-current          # debe decir: desarrollo

# ── 2. Programar (en la otra terminal, con ionic serve) y guardar ──
git status                         # ver qué ha cambiado
git add .
git commit -m "Productos: añadir paginación"
git push origin desarrollo         # subir tu trabajo (todavía NO se publica
   en Vercel)

# ── 3. Cuando esté probado y bien: pasarlo a main ──
git checkout main                  # ir a la rama buena
git merge desarrollo               # traer los cambios de desarrollo
git push origin main               # subir main → Vercel publica
   automáticamente

# ── 4. Volver a desarrollo para seguir ──
git checkout desarrollo
```

Mensajes de commit: que digan **qué** has hecho, en presente y concretos: `"Productos: añadir columna de dimensiones"`, no `"cambios"` ni `"asdf"`. El profe los mira ("commits descriptivos" en la checklist).

21.6 Merge: qué puede pasar

- **Fast-forward** (lo normal en tu caso): si `main` no ha cambiado desde que creaste `desarrollo`, git simplemente "adelanta" `main` hasta donde está `desarrollo`. Sin preguntas.

- **Merge commit**: si las dos ramas tienen commits distintos, git crea un commit nuevo que junta ambas. Puede abrirte un editor para el mensaje (en la terminal: escribe `:wq` y Enter si es Vim).

- **Conflicto**: si las dos ramas cambiaron **las mismas líneas** del mismo fichero, git no sabe cuál quedarse y marca el fichero así:

  ```
  <<<<<<< HEAD
  <ion-title>Productos</ion-title>          ← versión de la rama actual
  (main)
  =======
  <ion-title>Catálogo</ion-title>           ← versión de desarrollo
  >>>>>>> desarrollo
  ```

  Solución: editar el fichero dejando la versión buena (borrando las marcas `<<<<<<<`, `=======`, `>>>>>>>`), luego `git add .` y `git commit`. VS Code te pone botones *"Accept Current / Incoming / Both"* encima del conflicto. Si trabajas siempre en `desarrollo` y solo haces merge hacia `main`, casi nunca te pasará.

Alternativa desde la web: en GitHub, botón **"Compare & pull request"** → de `desarrollo` a `main` → **Merge pull request**. Hace lo mismo que el `git merge`, pero queda registrado en la web. Después, en tu PC: `git checkout main` + `git pull` para tener el `main` actualizado.

21.7 GitHub y Vercel con ramas

- **En GitHub**: el selector de ramas (arriba a la izquierda, pone `main`) permite ver el código de cada rama. El profe puede mirar `desarrollo` (en lo que estás) o `main` (lo validado).
- **En Vercel**: *Settings → Git → Production Branch* = `main`. Cada push a `main` = **despliegue de producción** (la URL oficial). Cada push a `desarrollo` = **despliegue de vista previa** (*Preview*, con una URL rara), útil para probar antes de pasar a `main`.
- Tu repo tiene además dos ramas viejas, `develop` y `prueba`, de las primeras semanas. Ya no se usan; se pueden borrar más adelante desde GitHub (*Branches* → icono de papelera).

21.8 `push`, `pull` y `fetch`

Comando	Qué hace
`git push`	Sube tus commits a GitHub
`git pull`	Baja de GitHub los commits que no tienes y los mezcla con tu rama (útil si trabajas en dos PCs o si cambiaste algo desde la web)
`git fetch`	Solo consulta qué hay nuevo en GitHub, sin mezclar nada
`git push --force`	**Sobrescribe** GitHub con tu versión, borrando lo que hubiera allí y tú no tengas. Peligroso: solo cuando sabes exactamente por qué (lo usamos una vez para separar las asignaturas en repos distintos).

21.9 Problemas típicos y cómo resolverlos

Mensaje	Qué significa	Solución
`fatal: not a git repository`	No estás dentro de un repo (p. ej. en `C:\DAM 2`)	`cd "C:\DAM 2\Desarrollo de Interfaces"`
`nothing to commit, working tree clean`	No hay cambios (o ya hiciste commit)	Nada: está todo guardado. ¿Te falta el `push`?
`The current branch desarrollo has no upstream branch`	Es el primer push de esa rama	`git push -u origin desarrollo`
`! [rejected] ... (fetch first)` / `non-fast-forward`	GitHub tiene commits que tú no tienes	`git pull` y luego `git push`
`Your local changes would be overwritten by checkout`	Cambios sin commit al cambiar de rama	`git add .` + `git commit` antes del `checkout`
`Unable to create '.../index.lock': File exists`	Un proceso de git se quedó a medias	Cerrar otros git/VS Code; borrar el fichero `.git\index.lock`
`warning: LF will be replaced by CRLF`	Aviso de saltos de línea Windows/Linux	Inofensivo, ignorarlo
`CONFLICT (content): Merge conflict in ...`	Las dos ramas cambiaron las mismas líneas	Ver 21.6

21.10 Chuleta de bolsillo

```
cd "C:\DAM 2\Desarrollo de Interfaces"   # SIEMPRE primero: entrar en el
  repo
git status                               # ¿qué hay cambiado? ¿en qué rama
  estoy?
git checkout desarrollo                  # trabajar en desarrollo
git add .                                # preparar todo
git commit -m "Qué he hecho"             # foto local
git push origin desarrollo               # subir desarrollo
```

```
git checkout main                       # ─┐
git merge desarrollo                    #  ├ publicar: desarrollo → main → Vercel
git push origin main                    # ─┘
git checkout desarrollo                 # volver a trabajar
git log --oneline --graph --all         # ver el árbol de commits
```